

ANTONIUS' HANDBOOK

Useful Formulas, Constants, Units and Definitions Volume II - Programmers Paradise Version 0.046

Compiled by: Antonius William Torode
Written in: L^AT_EX



© 2026 Antonius Torode
All rights reserved.

This work may be distributed and/or modified under the conditions of Antonius' General Purpose License (AGPL).

The original maintainer of this work is: Antonius Torode.

The current maintainer of this work is: Antonius Torode.

Published by Antonius Torode.

Hosted at: <https://torodean.github.io/AHandbook.html>

Github Repository: <https://github.com/torodean/Antonius-Handbook-II>

Most Current Revision Date (Version 0.046): July 20, 2026

Torode, A.
Antonius' Handbook.
Michigan State University –
Department of Physics & Astronomy.
2020, Graduate.
Volume II.
Version: 0.046

Preface

This document is a compilation of useful programming formulations, definitions, constants, and general information used throughout my own schooling and research as a reference while furthering education. It's purpose is to provide a complete 'encyclopedia' per se of various codes, syntax and significant ideas used often. The idea and motivation behind it is to be a quick reference providing easily accessible access to necessary information for either double checking or recalling proper formulations or algorithms for use in various situations due to my own shortcomings in matters of memorization. All the material in this document was either directly copied from one of the references listed at the end or derived from scratch. On occasion typos may exist due to human error but will be corrected when discovered.

The version number is updated every time the document is modified. This ensures that there is no two copies with different information and identical version numbers. The latest update date is automatically set to the current date each time the document is edited. Please refrain from distributing this handbook without permission from the original author/compiler. This book is formatted for printing.

For more information about this book or details about how to obtain your own copy please visit:

<https://torodean.github.io/AHandbook.html>

Disclaimer

This book contains codes, formulas, definitions, and theorems that by nature are very precise. Due to this, some of the material in this book was taken directly from other sources. This is only such in cases where a change in wording or codes could cause ambiguities or loss of information quality. Following this, all known sources used are listed in the references section.

This page intentionally left blank.
(Yes, this is a contradiction.)

Contents

1	Introduction	1
2	Linux	2
2.1	System Related Commands	2
2.1.1	Checking RPM Packages and Versions	3
2.2	Remote Connections (SSH)	4
2.3	Files and Storage	5
2.4	Users and Groups	6
2.5	Networking	6
2.5.1	Redirecting program output	7
2.6	Shell/Bash Scripting	7
2.7	Other	10
2.7.1	emacs	10
3	Windows	11
4	Mac	12
5	C/C++	13
5.1	Data Types	13
5.2	Basics of the Language	13
5.2.1	Basic Input and Output	15
5.2.2	Variable Types	16
5.2.3	Class Structures	16
5.2.4	Converting Between Types	17
5.3	Mathematical Commands	18
5.4	System Commands	19
5.4.1	Simulate Key Strokes	19
5.5	Compiler/Processor specific	21
5.6	Advanced Coding Features	21
5.7	PImpl Idiom (Pointer to Implementation)	24
5.7.1	Example: PImpl Idiom	24
5.7.2	Benefits of PImpl	25
5.7.3	Considerations	25
5.8	Tag Dispatch	25
5.8.1	Example: Tag Dispatch	26
5.8.2	Benefits of Tag Dispatch	26
5.8.3	Considerations	27
5.9	Modern C++ Utility Types	27
5.9.1	std::optional	27
5.9.2	std::make_pair and std::tuple	27
5.9.3	std::apply	28
5.9.4	std::tie	29
5.9.5	std::ignore	29
5.9.6	std::variant	29
5.9.7	std::any	29
5.10	Smart Pointers	30
5.10.1	std::unique_ptr	30
5.10.2	std::shared_ptr	30
5.10.3	std::weak_ptr	31
5.10.4	std::make_unique and std::make_shared	31
5.11	Type Aliases	31
5.12	Range-Based For Loops	32

5.13	Lambdas	32
5.14	Structured Bindings	33
5.15	Conditional Compilation with <code>std::enable_if_t</code>	33
5.16	Variadic Templates and Fold Expressions	34
5.16.1	Variadic Templates	34
5.16.2	Parameter Pack Expansion	35
5.16.3	Fold Expressions	35
5.16.4	Perfect Forwarding	36
5.16.5	Summary of Variadic Template Features	36
5.17	Lvalues and Rvalues	36
5.18	Move Semantics	37
5.19	Virtual Functions and the Vtable	37
5.20	RAII (Resource Acquisition Is Initialization)	38
5.21	SFINAE (Substitution Failure Is Not An Error)	38
5.22	<code>auto</code> and <code>decltype</code>	39
5.23	<code>constexpr</code> and <code>constexpr</code>	39
5.24	<code>nullptr</code>	39
5.25	User-Defined Type Conversions	40
6	Make/CMake	41
6.1	CMake	41
7	Git	43
7.0.1	Basic Git	43
7.0.2	Advanced Git	44
7.0.3	Git Submodules	45
7.1	Github.io	46
7.2	Quality Of Life Hacks And Scripts	47
7.2.1	Repository Updating	47
8	L^AT_EX	48
8.1	Hello World	48
8.2	L ^A T _E X Basics	48
8.2.1	LaTeX Titles	48
8.3	LaTeX elements	49
9	C#	51
9.1	Useful Application Functions	51
9.2	Getting Windows System Information	52
10	Java	55
10.1	Java Basics	55
11	HTML	56
11.1	HTML Basics	56
11.2	Useful HTML scripts	57
12	CSS	58
12.1	CSS Basics	58
13	PHP	60
13.1	PHP Basics	60
14	JavaScript	61

15 Python	62
15.1 Basics of Python	62
15.1.1 Imports and libraries	62
15.1.2 Program parameters	62
15.1.3 Methods, Functions, and Classes	63
15.1.4 String manipulation	64
15.1.5 Arrays and lists	64
15.1.6 Plotting and Graphs	65
15.2 Pytest	66
15.3 Python tk GUI	67
15.4 Useful Methods and Packages	68
15.4.1 Useful Packages	68
15.4.2 Useful Methods	74
16 Resources	79
References	79

Introduction

This handbook is a comprehensive collection of programming tips, snippets, notes, and more, curated by programmers for programmers. Whether you're a seasoned developer looking for quick references or a beginner seeking valuable insights, this handbook is designed to be your go-to resource. From C++ to HTML, LaTeX to Python, CMake, and beyond, Programmers Paradise covers a wide range of languages and topics. Its primary aim is to serve as a personal repository for quick retrieval of essential information, but its organized format and extensive content make it equally valuable for anyone in the programming community.

Linux

Linux is a broad subcategory that encompass a large family of free and open sourced operating systems. Installing, setting up, and using a Linux based operating system is the perfect way for anyone to gain knowledge, understanding, and practice of how a computer system truly works. Unlike the end user experience with Windows and Mac OS, Linux has a much higher capability for customization and a higher degree of freedom. With that said, Linux is not necessarily more user friendly to the new or average computer user, however it is free in most cases!

2.1 System Related Commands

Retrieve information and valid arguments for a command. This works with many commands.

```
1 COMMAND --help # COMMAND must be a valid command such as cd, ls, etc...
```

To view the **history** of commands entered or **delete** a history entry, you can use the **history** command. This could be useful if you accidentally typed in your **password** or something similar into the terminal.

```
1 history # View command history.
2 history -d <ID> # Delete the history entry listed with number <ID>
3 # Prepend a space to a command to exclude it from history (if HISTCONTROL=
  ↪ ignorespace).
```

Installing and updating packages using package managers such as **apt**, **yum**, or **dnf**.

```
1 sudo apt update # Updates the list of available packages (APT)
2 sudo apt upgrade # Upgrades installed packages to their latest versions (APT)
3 sudo yum update # Updates the list of available packages (Yum)
4 sudo yum upgrade # Upgrades installed packages to their latest versions (Yum)
5 sudo dnf update # Updates the list of available packages (DNF)
6 sudo dnf upgrade # Upgrades installed packages to their latest versions (DNF)
```

Monitoring system resource usage using tools like **top**, **htop**, and **iotop**.

```
1 top # Displays dynamic real-time information about running processes
2 htop # An interactive process viewer
3 iotop # Displays I/O usage by processes in real-time
```

Changing directory via terminal via the **cd** command.

```
1 cd /directory # Changes the directory to the subdirectory /directory
2 cd .. # Goes back one directory
```

Getting **current directory** via terminal

```
1 pwd
```

How to display the processes that are currently running.

```
1 ps aux
```

To search the results of a command for a string of characters one can use the **grep** command. For example:

```
1 ps aux | grep "firefox"
```

Restore power/battery icon if it disappears on a laptop.

```
1 /usr/lib/x86_64-linux-gnu/indicator-power/indicator-power-service &disown
```

Restore volume icon/control button if it disappears.

```
1 gsettings set com.canonical.indicator.sound visible true
```

The command-line utility that is used to control systemd is called **systemctl**, which is the init system and service manager in modern Linux distributions.

```
1 sudo systemctl start <service_name>.service      # Start a service.
2 sudo systemctl stop <service_name>.service       # Stop a service.
3 sudo systemctl status <service_name>.service     # Get Status of a service.
4 sudo systemctl restart <service_name>.service    # Restart a service.
5 sudo systemctl enable <service_name>.service     # Enables a service to start on
   ↳ boot
6 sudo systemctl disable <service_name>.service   # Disables a service from
   ↳ starting on boot
```

Reset **wifi** services in case the connection gets lost.

```
1 sudo systemctl restart network-manager.service
```

The command-line utility used for querying and displaying **logs** from the systemd journal is **journaltl**. The **-e** flag will start the log at the end (most recent) logged messages, which is often useful.

```
1 sudo journalctl -e # Jumps to most recent logs
2 sudo journalctl -xe
```

Turn off LCD display.

```
1 xset dpms force off // Turns off display.
```

Change or view the **host** name of a computer with the **hostname** file.

```
1 sudo nano /etc/hostname # Opens this file using nano for editing.
2 hostname               # Command to see what the current hostname is.
3 sudo hostnamectl set-hostname <name> # Sets hostname without a needed reboot.
```

To get the **date** and **time** of a system, you can use the **date** command. You can further specify parameters to only return a single component of this which may be useful in scripting.

```
1 date                # Returns "Tue Apr  2 13:31:21 UTC 2024"
2 date +%Z            # Returns "UTC"
3 date +%Y            # Returns "2024"
```

To run a command continuously on a loop and view the output, you can use the **watch** command. This does not typically preserve colors in the output.

```
1 watch -n 5 <command>          # Runs <command> every 5 seconds.
2 watch --color -n 5 <command> # Attempts to preserve colors in output
```

2.1.1 Checking RPM Packages and Versions

To query and manage RPM (Red Hat Package Manager) packages installed on a system, you can use the **rpm** command with various options to retrieve information such as package versions, details, and installation status.

```
1 rpm -q <package>                # Queries if a package is installed.
2 rpm -q --queryformat '%{VERSION}-%{RELEASE}' <package> # Displays the version
   ↳ and release of the specified package.
3 /usr/bin/rpm -q --queryformat '%{VERSION}-%{RELEASE}' <package> # Same as above
   ↳ , using the full path to the rpm binary.
```

```
4 rpm -qi <package> # Displays detailed information about the
    ↳ specified package, including version, release, and description.
5 rpm -qa # Lists all installed RPM packages on the
    ↳ system.
6 rpm -qa | grep <package> # Searches for a specific package in the list
    ↳ of all installed packages.
7 rpm -qf /path/to/file # Identifies which package owns a specific file
    ↳ on the system.
8 rpm -q --changelog <package> # Shows the changelog of the specified package.
```

2.2 Remote Connections (SSH)

Accessing remote systems using **Secure Shell (SSH)**.

```
1 ssh username@hostname # Connects to the remote system using SSH
```

Transferring files securely between systems using **scp** or **sftp**.

```
1 # Copies a file from local to remote system
2 scp /path/to/local/file username@hostname:/path/to/remote/location
3
4 # Copies a file from remote to local system
5 scp username@hostname:/path/to/remote/file /path/to/local/location
6
7 # Initiates interactive SFTP session with remote system
8 sftp username@hostname
```

Accessing remote desktops using **X11 forwarding** and X terminal.

```
1 # Enables X11 forwarding for remote system
2 ssh -X username@hostname
3
4 # Opens a new X terminal window from remote system
5 xterm
```

Securely transferring files between systems using **rsync**.

```
1 # Synchronizes files and directories between local and remote systems
2 rsync -avz /source/path username@hostname:/destination/path
```

Accessing remote systems via graphical interface using tools like **VNC** or **XRDP**.

```
1 # Opens a VNC session to remote system
2 vncviewer hostname
3
4 # Opens a Remote Desktop Protocol (RDP) session to remote system
5 rdesktop hostname
```

Establishing persistent remote connections using tools like **tmux** or **screen**.

```
1 tmux new -s sessionname # Creates a new tmux session
2 screen -S sessionname # Creates a new screen session
```

2.3 Files and Storage

To find a file within a folder or its sub-folders, you can use the **find** command.

```
1 find -name "fileName.txt" # Finds a file named fileName.txt
2 find -name "file*" # Finds a file containing "file" in its name.
```

Copy a file or directory to a different computer

```
1 # To copy a file.
2 scp -v <File Path> username@computer:"<path to copy to>"
3
4 # To copy a directory.
5 scp -rv <File Path> username@computer:"<path to copy to>"
```

Show information about the file system on which each FILE resides, or all file systems by default.

```
1 df
```

Retrieve the Disk Usage (file sizes) of a directory or its contents.

```
1 du # List the size of the subdirectories.
2 du -sh # List the size of the directory in a human readable format.
3 du -ah # Lists the size of all files in the directory.
```

List information about File(s) (in the current directory by default). You can specify the number of entries you want listed using the **head** and **tail** commands.

```
1 ls # list all items in a directory
2 ls -l # list all items in a directory (one item per line)
3 ls -lh # list all items with size, owner, and date modified
4 ls -lu # list all items with size, owner, and date accessed.
5 ls -lc # list all items with size, owner, and date of status change.
6 ls -lhrt # Useful ls output
7
8 ls -lhrt | head -4 # Outputs only the first four entires
9 ls -lhrt | tail -4 # Outputs only the last four entires
```

The **stat** command is used to display detailed information about a file or directory. It provides various details such as file type, permissions, size, timestamps (access, modify, and change), and more.

```
1 stat filename
```

To list all of the **block devices** (and hence **partitions**) detected by the machine, you can use the '**lsblk**' command. Block devices are physical or **virtual** devices that can be used for data storage, such as hard drives, SSDs, and partitions.

```
1 lsblk
```

To **mount** and **unmount** a partition

```
1 sudo mount <DEVICE TO MOUNT> <MOUNT POINT>
2 sudo mount /dev/sdb1/ /mnt/ # example of mounting
3 sudo umount <DEVICE TO MOUNT> <MOUNT POINT>
4 sudo umount /dev/sdb1/ /mnt/ # example of mounting
```

To open a **pdf** via terminal, most generic desktop environments support

```
1 xdg-open filename.pdf
```

Creating backups using tools like **tar**, **rsync**, or **backuppc**.

```

1 # Creates a compressed tarball of a directory
2 tar -czvf backup.tar.gz /path/to/directory
3
4 # Synchronizes files and directories between two locations
5 rsync -avz /source/path /destination/path

```

2.4 Users and Groups

List all users

```

1 cut -d: -f1 /etc/passwd

```

Create a new user using the **useradd** command.

```

1 sudo useradd [options] <USERNAME>      # Creates a user.
2 sudo useradd -e 2016-02-05 <NAME>      # Creates a user that expires on a day.
3 sudo useradd <USERNAME> -G <GROUPNAME> # Adds a user to a group upon creation.
4 useradd --help                          # See full useradd options.

```

Change a users password using **passwd**.

```

1 passwd <USERNAME>

```

Change the user in terminal using the **su** command.

```

1 su - <USERNAME>

```

Add a user to the sudoers group

```

1 usermod -aG sudo <USERNAME>

```

Managing user permissions and file ownership using **chmod**, **chown**, and **chgrp**.

```

1 chmod 755 file      # Changes file permissions to read, write, and execute
   ↪ for owner, and read and execute for group and others.
2 chmod a+x file      # Changes file permissions to include execute permissions
   ↪ .
3 chown user:group file # Changes the owner and group of the file.
4 chgrp group file      # Changes the group of the file.

```

2.5 Networking

The **ifconfig** command is for viewing IP configuration information and configuring network interface parameters.

```

1 ifconfig

```

The **traceroute** command is for printing the route that packets take to a network host.

```

1 traceroute

```

The **Domain Information Groper** is used to perform DNS lookups and display answers returned from the DNS servers.

```

1 dig

```

The **telnet** command connects the destination host:port via the telnet protocol. An established connection means connectivity between two hosts is properly working.

```
1 telnet
```

The **nslookup** command is for querying Internet domain name servers.

```
1 nslookup
```

The **netstat** command is used to review open network connections and open sockets.

```
1 netstat
```

The **nmap** command is used to check for opened ports on a server

```
1 nmap <SERVER NAME>
```

The **ifup** and **ifdown** commands are used to disable network interfaces.

```
1 # Enables an ethernet parameter
2 ifup <ETHERNET INTERFACE PARAMETER>
3 ifup eth0 # example: enables 'eth0'
4
5 # Disables an ethernet parameter
6 ifdown <ETHERNET INTERFACE PARAMETER>
7 ifdown eth0 # example: disables 'eth0'
```

Enable/Disable **IPv6**. This is only a temporary solution as it may turn itself back on after some time.

```
1 # Use these two commands to disable IPv6
2 sudo sysctl -w net.ipv6.conf.all.disable_ipv6=1
3 sudo sysctl -w net.ipv6.conf.default.disable_ipv6=1
4
5 # Use these two commands to re-enable IPv6
6 sudo sysctl -w net.ipv6.conf.all.disable_ipv6=0
7 sudo sysctl -w net.ipv6.conf.default.disable_ipv6=0
```

2.5.1 Redirecting program output

When outputting to a file, there is an append and an overwrite operator.

```
1 command > output.txt // Writes output to output.txt
2 command >> output.txt // Appends output to output.txt
```

To redirect all output from a program (including **stdout** and **stderr**), you can use

```
1 command -args > output.txt 2>&1
```

2.6 Shell/Bash Scripting

To create a shell script you must create a new text file and save it as a '.sh' file. The file should start with the directory to the proper shell which is generally the default below. The first line (starting with a **shebang** '#!') is not a comment, but instead is treated by Unix as "which shell do I use to run this code." In our case, the Bourne shell will be used [5]. Furthermore, to create a shell application that has parameters, with a help screen to explain those parameters, you can apply the following template.

```

1  #!/bin/sh
2  # This is a comment!
3
4  # This creates a method to print the usage of the script
5  usage()
6  {
7      cat << EOF
8      purpose: This explains the purpose of the script
9      Usage:   $0 [opts]
10
11  OPTIONS:
12      -h          Display this help message
13      -f <file>   File to input
14      -v          Boolean-type flag as parameter
15
16  EOF
17
18
19  # This parses the arguments input to the script. The ":" specifies a parameter
20  ↪ is expected with the input flag.
21  while getopts "hf:v" OPTION; do
22      case $OPTION in
23          h)  usage          # Calls out usage method.
24              exit 0         # Exits with error code 1 => success.
25              ;;             # Ends the specified case
26          f)  fileInput=$OPTARG
27              ;;
28          v)  verboseMode=1  # Sets verboseMode to true.
29              ;;
30          *)  echo "Invalid option entered: -$OPTARG" >&2
31              usage
32              exit 2         # Exits with error code 2 => fail.
33              ;;
34      esac                # completes out case statement
35  done
36
37  # This checks if a flag was used.
38  if [[ $verboseMode ]]; then
39      echo "Verbose mode is activated!"
40  elif [[ ! $verboseMode ]]; then
41      echo "Verbose mode is disabled!"
42  fi
43  }

```

To print text one can use the **echo** command as follows.

```

1  #!/bin/sh
2  echo Hello World
3  echo "Hello World"
4  echo -e "In order to print newline characters, use the e option!\n"

```

To make a file executable, or change the permissions in general the **chmod** command can be used and is typically used as follows.

```

1  chmod a+x <SCRIPTNAME>.sh          # Make a script executable.

```



```

2  chmod -R 775 filesToChangePermissionsOf.ext # Change the permissions of some
    ↪ file.

```

To modify the ownership of files you can specify the owner and group of a file(s) using **chown**

```

1  chown username:group file.txt # Change the owner and group of a file.
2  chown -R :group folder      # Change only the group of some folder and its
    ↪ sub-folders.

```

Shell script **variables** are created by use of the equal sign. spaces in lines containing variables need to be avoided. To reference a variable, the '\$' character is used. Quotations are used to avoid ambiguities with spaces.

```

1  #!/bin/sh
2  MY_VARIABLE="Hello World" # Creates a variable.
3  echo $MY_VARIABLE        # Prints the variable.

```

To use a variable within a terminal session, you can use **export** it to store it for that session.

```

1  export PATH="$PATH:/home/user/.local/bin/" # Appends the PATH variable with a
    ↪ string

```

The **touch** command can be used to create a new empty file.

```

1  #!/bin/sh
2  echo "What is your name?"
3  read USER_NAME
4  echo "Hello $USER_NAME"
5  echo "I will create you a file called ${USER_NAME}_file"
6
7  # The quotations prevent multiple files from being called to touch.
8  touch "${USER_NAME}_file"

```

To determine the total number of files of a given file patter, you can use the **wc** command to count the output of an **ls** command. To save the output of a script or command, you must enclose it within the correct elements of '\$()' such as below.

```

1  totalFiles=$(ls -l $filePattern 2> /dev/null | wc -l)

```

To perform **arithmetic** within shell scripts, you must use double parenthesis.

```

1  firstVar=4
2  secondVar=7
3  addedVar=$((firstVar+secondVar))

```

To create a **for loop** in shell, you can do so like the following:

```

1  i=1
2  for file in $filePattern; do
3      [[ ! -e $file ]] && continue
4      echo "I see file number $i: $file"
5      ((i++))
6  done

```

Bash provides a feature called "**brace expansion**," which allows you to generate arbitrary strings based on patterns using curly braces . While this feature is commonly used to generate sequences of numbers or letters, it can also be used in conjunction with command-line shortcuts to save time and improve efficiency.

```

1  # This will create directory1, directory2, ... and directory5.
2  mkdir directory{1..5}

```

2.7 Other

The **mkfifo** command is used to create a new named pipe. A pipe is used to store the output of one program to be used in another.

```
1 mkfifo namedPipe    # Creates a pip named "namedPipe".
2 ls > namedPipe      # Feeds the output of ls into namedPipe.
3 cat < namedPipe     # Feeds namedPipe into cat and displays the data from ls.
4 mkfifo namedPipe2 -m700 # Modifies the permissions of a created pipe.
```

2.7.1 emacs

Emacs is a powerful text editor. You can open a document through emacs using the following. The “-nw” flag indicates no GUI window should open (open in terminal).

```
1 emacs doc.txt -nw
```

To toggle **read-only mode** use C-x C-q.

Windows

3.1: Windows Key Combinations

Windows has various key combinations. These are used to do different things. For the purposes of this chart, the windows key will be represented by .

Key combination	Descriptions
 + R	Opens a run window.
 + M	Minimize all open windows.
 + E	Opens the explorer window.
 + UP	Minimize the currently opened window.
 + F	Opens search for searching files and folders.
 + P	Opens display projection settings.
Alt + Tab	Change between open windows.
CTRL + ALT + Delete	Provides user options such as changing password.
CTRL + SHIFT + ESC	Opens Windows Task Manager.

To access the Windows 7 "**God Mode**" which is essentially a collection of administrator and troubleshooting features, create a folder with the following name:

```
1 GodMode.{ED7BA470-8E54-465E-825C-99712043E01C}
```

To view system information, including RAM installed, graphics processor and more, run the following command

```
1 dxdiag
```

To view and manage the **services** that are running on a machine, you can access the services.msc application by opening a run window and entering

```
1 services.msc      # Opens the services running.
```

To view and modify **mouse** settings, such as sensitivity or speed, you can open a run window and enter

```
1 main.cpl          # Opens the mouse settings.
```

The **System File Checker** is a built-in tool that scans for and restores corrupted system files. You can run it from the Command Prompt with administrative privileges.

```
1 sfc /scannow
```

Mac

MacOS is Apple’s proprietary Unix-based operating system tailored for Macintosh computers. It boasts a sleek graphical user interface (GUI) atop a Unix-like foundation, offering stability and security. Its software ecosystem encompasses a broad array of third-party applications accessible through the Mac App Store and other channels. Notable features include seamless integration with Apple’s ecosystem, such as iCloud and iOS devices, robust security measures like Gatekeeper and FileVault for app and disk encryption, respectively, and advanced privacy controls. MacOS also provides essential productivity tools and development support, including Xcode and Terminal, making it an appealing platform for software engineers and developers.

4.1: Mac Startup Options

Mac has various startup features. To use them, hold the following keys down simultaneously upon startup as soon as you hear the startup chime:

Startup Keys	Descriptions
Command, R	Boot into OS X Recovery mode.
C	Boot to external device such as CD, DVD, or USB.
N	Netboot.
Shift	Safe Boot.
Command, V	Boot using verbose mode for comprehensive boot details.
Command, S	Single user mode.
Command, Option, P, R	Resetting the PRAM during boot.
T	Enable target disk mode.

C/C++

C++ is a powerful, high-level programming language renowned for its performance, flexibility, and extensive standard library. Developed by Bjarne Stroustrup in the early 1980s, C++ is an extension of the C programming language with added support for **object-oriented** programming (**OOP**) features, such as **classes**, **inheritance**, and **polymorphism**. C++ is widely used in systems programming, game development, embedded systems, and performance-critical applications due to its close-to-the-metal capabilities and efficient **memory management**. Its rich standard library provides developers with a wide range of functions and data structures for building robust and scalable applications. C++’s versatility allows developers to write code that is both low-level, allowing direct hardware access, and high-level, facilitating complex software design patterns. Despite its complexity, C++ remains a popular choice for developers seeking performance and control over their software projects.

5.1 Data Types

5.1: C Integer data types

This information is taken from ??

C type	stdint.h type	Bits	Sign	Range
char	uint8_t	8	Unsigned	0 .. 255
signed char	int8_t	8	Signed	-128 .. 127
unsigned short	uint16_t	16	Unsigned	0 .. 65,535
short	int16_t	16	Signed	-32,768 .. 32,767
unsigned int	uint32_t	32	Unsigned	0 .. 4,294,967,295
int	int32_t	32	Signed	-2,147,483,648 .. 2,147,483,647
unsigned long long	uint64_t	64	Unsigned	0 .. 18,446,744,073,709,551,615
long long	int64_t	64	Signed	-9,223,372,036,854,775,808 .. 9,223,372,036,854,775,807
C type	IEE754 Name	Bits		Range
float	Single Precision	32		-3.4E38 .. 3.4E38
double	Double Precision	64		-1.7E308 .. 1.7E308

5.2 Basics of the Language

A basic C++ **hello world** program follows. This program includes the `<iostream>` header file, which provides input and output functionality. The `main()` function is the entry point of the program, and it simply outputs "Hello, World!" to the console using `std::cout`, which is the standard output stream. Finally, `return 0;` indicates successful program execution.

```
1 #include <iostream>
2
3 int main() {
4     // Print "Hello, World!" to the console
5     std::cout << "Hello, World!" << std::endl;
6     return 0;
7 }
```

The C++ main function is designed by default to pass in **arguments** when a program is ran. The argument passing is set up as follows.

```
1 int main(int argc, char *argv[])
2 {
```

```

3 // argc would represent how many arguments were passed to the program.
4 // argv[] is an array of each argument with the first element being the program name.
5 for (int i=0;i<argc;i++) std::cout << argv[i] << std::endl; //Prints the arguments.
6 return 0;
7 }
8
9 // So for example if I had HelloWorld.cpp as the above and ran using
10 // ./HelloWorld argument1 argument2 argument3
11 // we would get an output of
12 /some/file/path/HelloWorld
13 argument1
14 argument2
15 argument3

```

“The **address** of a variable can be obtained by preceding the name of a variable with an **ampersand** sign (&), known as **address-of** operator. [1]

```

1 var = 314; //Creates a variable and stores in in memory.
2 address = &var; //Returns the memory address of the stored variable.

```

“**Pointers** are said to ‘point to’ the variable whose address they store.” [1] Proceeding a pointer with the dereference operator (*), which can be read as ‘value pointed to by’ can be used to access a variable which stores the address of another variable (called a **pointer**).

```

1 pointer = *address; //Sets pointer to the value of the variable that address points to.

```

A pointer must be declared using the type of the data the pointer points to.

```

1 int * number; //Creates number to point to an int.
2 char * character; //Creates character to point to a char.
3 double * decimals; //Creates decimals to point to a double.
4 int array [20]; //Creates an empty array with 20 elements.
5 number = &array[2]; //Sets number to point to the third memory slot of array.
6 cout << *number; //Prints the value stored in array[2].

```

Incrementing pointers. “When adding one to a pointer, the pointer is made to point to the following element of the same type, and, therefore, the size in bytes of the type it points to is added to the pointer.”

```

1 char *mychar; //Creates mychar to point to a char.
2 short *myshort; //Creates myshort to point to a short.
3 long *mylong; //Creates mylong to point to a long.
4
5 ++mychar; //Would increment to the next memory slot (sizeof(char) = 1 byte).
6 ++myshort; //Would increment two memory slots (sizeof(short) = 2 bytes).
7 ++mylong; //Would increment sizeof(long) bytes: 4 on Windows (LLP64), 8 on most
  ↳ Linux/macOS 64-bit (LP64).
8
9 *p++ // same as *(p++): increment pointer, and dereference unincremented address
10 **p // same as *(++p): increment pointer, and dereference incremented address
11 ++*p // same as ++(*p): dereference pointer, and increment the value it points to
12 (*p)++ // dereference pointer, and post-increment the value it points to

```

Within C++, you can use operations with **pointers to functions** which is typically used when calling a function with another function as a parameter. An example follows as [1]:

```

1 int addition (int a, int b){ return a+b; }
2 int subtraction (int a, int b){ return a-b; }
3
4 int operation (int x, int y, int (*functocall)(int,int)){
5     int g;
6     g = (*functocall)(x,y);

```

```

7     return g;
8 }
9
10 int main (){
11     int m,n;
12     int (*minus)(int,int) = subtraction; //minus is a pointer to a function that has two
        ↪ parameters of type int.
13
14     m = operation (7, 5, addition);
15     n = operation (20, m, minus);
16     cout << n;
17     return 0;
18 }

```

Templates can be used for defining classes that support multiple types.

```

1 template<class type>
2 class className{ //Creates a class named className
3     type a,b;    //Creates some variables of type
4 public:
5     className(type val1, type val2) : a(val1), b(val2){}; //Constructor for className.
6     type getMax(){ return a>b ? a:b; };
7 };

```

...Alternately, the above code can also be written as below. Note that this is a *member-function definition*, not a free function: because it is a member of `className`, it may access the private members `a` and `b`. For a class template like this, the out-of-line definition must also appear in the same header (visible at the point of instantiation), not in a separate `.cpp` file.

```

1 template<class type>
2 class className{ //Creates a class named className
3     type a,b;    //Creates some variables of type
4 public:
5     className(type val1, type val2){ a = val1; b = val2; }; //Constructor for className.
6     type getMax();
7 };
8
9 template<class type>
10 type className<type>::getMax(){ return a>b ? a:b; }

```

To loop over **boolean** values, you can use

```

1 for (bool a : { false, true }) { /* ... */ }

```

5.2.1 Basic Input and Output

To output text via a terminal you can use:

```

1 uint32_t number = 0x123456 // A hexadecimal number.
2 std::string text = "Hello World!"; // A string.
3
4 std::cout << text << std::endl; // std::endl is equivalent to the new-line character.
5 std::cout << std::hex << number; // Prints a number in hexadecimal format.

```

To get input as a user in the type of a `std::string`, you can use:

```

1 std::string input = "";
2 std::cout << "Enter some text: ";
3 std::getline(std::cin, input);

```

5.2.2 Variable Types

Creating and using a vector.

```

1  #include <vector>
2
3  int size1 = 5;
4  int size2 = 6;
5  uint32_t number = 0x345678; //Creates a 32 bit unsigned integer and sets it in
    ↪ hexadecimal.
6
7  //Creates a vector named V1 containing int's with a size of 5 and sets each element to 0.
8  std::vector<int> V1(size1, 0);
9
10 //Creates a 2-D vector (vector containing vectors) of size 5x6 named V2 containing
    ↪ doubles;
11 std::vector< std::vector<double>> V2(size1, std::vector<double>(size2, 0));
12
13 V1[0] = 8; //Sets the first element in V1 to 8.
14
15 V2[0][3] = 3.1415; //Sets the 4th element in the first row of V2 to 3.1415.

```

5.2.3 Class Structures

In C++ a **Class** is an object that can contain variables and functions all defined within the object to be used in various ways. Classes are instantiated by a **constructor** and torn down by a **destructor**.

```

1  // This creates a class named ParentClass.
2  class ParentClass{
3      // The public members of a class are accessible to anything outside of the class.
4      public:
5          ParentClass(); // Constructor for the ParentClass.
6          ~ParentClass(); // Destructor for the ParentClass.
7          int notSoSpecialInt = 13; // Creates an integer.
8  };
9
10 // This creates a class named ChildClass and inherits the public features of another
    ↪ class ParentClass.
11 class ChildClass : public ParentClass {
12     // The public members of a class are accessible to anything outside of the class.
13     public:
14         // Creates a public method to return notSoSpecialInt.
15         int getNotSoSpecialInt() { return notSoSpecialInt; };
16         // Creates a public method to return secretInt.
17         int getSecretInt() { return secretInt; };
18
19     // The protected members of a class are accessible to this class and any class that
        ↪ inherits this one.
20     protected:
21         void hello(); // Creates a protected method hello() that is not defined.
22
23     // The private members of a class are only accessible to this class.
24     private:
25         int secretInt = 2; // Creates a private integer secretInt.
26 };
27
28 // This defines the hello method within the ChildClass class.
29 void ChildClass::hello() {
30     std::cout << "Hello!" << std::endl;

```



```

31 }
32
33
34
35 int main() {
36     ChildClass child; // Creates an object of ChildClass named child.
37
38     int number = child.secretInt(); // ERROR: This would not work because secretInt is
    ↪ private;
39     int number = child.getSecretInt(); // SUCCESS: This works because getSecretInt() is
    ↪ public.
40     return 0;
41 }

```

5.2.4 Converting Between Types

5.2.4.1 std::string to *

To convert a **std::string** to an **integer** you can use the **stoi** function:

```

1 std::string text = "31415";
2 int number = std::stoi(text);

```

To convert a **std::string** to a **double** you can use the **stod** function:

```

1 std::string text = "3.1415";
2 double number = std::stod(text);

```

To convert a **std::string** to a **const char*** you can use the **c_str** function:

```

1 std::string str = "3.1415";
2 const char* chr = str.c_str();

```

To convert a **std::string** to a **long long int** you can use the **stoll** function:

```

1 std::string text = "123456789012345";
2 long long int number = std::stoll(text);

```

To convert a **std::string** to a **float** you can use the **stof** function:

```

1 std::string text = "3.1415";
2 float number = std::stof(text);

```

To convert a **std::string** to a **unsigned int** you can use the **stoul** function:

```

1 std::string text = "12345";
2 unsigned int number = std::stoul(text);

```

To convert a **std::string** to a **long double** you can use the **stold** function:

```

1 std::string text = "3.14159265358979323846";
2 long double number = std::stold(text);

```

5.2.4.2 * to std::string

To convert various types back to a **std::string**, you can use the **std::to_string** method:

```

1 int number_int = 31415;
2 std::string text_int = std::to_string(number_int);
3
4 double number_double = 3.1415;

```

```

5 std::string text_double = std::to_string(number_double);
6
7 // For const char*, you can use the string constructor.
8 const char* chr = "3.1415";
9 std::string str_char(chr);
10
11 long long int number_ll = 123456789012345;
12 std::string text_ll = std::to_string(number_ll);
13
14 float number_float = 3.1415;
15 std::string text_float = std::to_string(number_float);
16
17 unsigned int number_uint = 12345;
18 std::string text_uint = std::to_string(number_uint);
19
20 long double number_ld = 3.14159265358979323846;
21 std::string text_ld = std::to_string(number_ld);

```

5.3 Mathematical Commands

5.3.0.1 Prime Numbers

A simple brute force method to determines if a number of type long is **prime** or not.

```

1 bool isPrime(long num) {
2     int c = 0;    //c is a counter for how many numbers can divide evenly into num
3     if (num == 0 || num == 1 || num == 4) {
4         return false;
5     }
6     for (long i = 1; i <= ((num + 1) / 2); i++) {
7         if (c < 2) {
8             if (num % i == 0) {
9                 c++;
10            }
11        } else {
12            return false;
13        }
14    }
15    return true;
16 }

```

5.3.0.2 Fibonacci Sequence

A simple brute force method for determining the **Fibonacci** numbers.

```

1 int fibonacci(int n) {
2     if (n <= 1) return n;
3     return fibonacci(n - 1) + fibonacci(n - 2);
4 }

```

5.3.0.3 Trigonometric Identities

To perform calculations using trigonometric identities, you first must include `cmath` and then do so as follows. These trigonometric functions from `cmath` can be used as floats, doubles, or long doubles.

```

1 #include <cmath>                                // Needed at start of file.
2
3 float num = 0.05;                                // creating a number.
4 float numS = std::sin(num);                      // Calculates the sin of the number
5 float numC = std::cos(num);                      // Calculates the cos of the number
6 float numT = std::tan(num);                      // Calculates the tan of the number

```

5.4 System Commands

Sleep

Make the thread **sleep** for some amount of time using the `std::chrono` to determine the duration [2].

```

1 #include <thread>
2 #include <chrono>
3
4 // Makes the system sleep for 50 milliseconds.
5 std::this_thread::sleep_for(std::chrono::milliseconds(50));
6
7 // Makes the system sleep for 50 seconds.
8 std::this_thread::sleep_for(std::chrono::seconds(50));

```

On a **Windows** specific program this can be simplified by including the `windows.h` header

```

1 #include <windows.h>
2
3 Sleep(50);    // Makes the system sleep for 50 milliseconds.
4 Sleep(5000); // Makes the system sleep for 50 seconds.

```

On a **Windows** specific program one can run a command directly from command prompt using the `system` function. The input variable to `system` is **const char***.

```

1 #include <windows.h>
2
3 //system(const char* input)
4 system("DATE"); // Runs the DATE command from windows command prompt.

```

5.4.1 Simulate Key Strokes

5.4.1.1 Key Strokes on Windows

First the correct files must be included and an event must be setup.

```

1 #define WINVER 0x0500
2 #include <windows.h>
3
4 INPUT ip;
5
6 ip.type = INPUT_KEYBOARD; // Set up a generic keyboard event.
7 ip.ki.wScan = 0; // hardware scan code for key.
8 ip.ki.time = 0;
9 ip.ki.dwExtraInfo = 0;

```

After this, functions can be setup to simulate various keys based on the specific key codes, two examples of such are

```

1 void space() {
2     // Press the "space" key.

```

```

3   ip.ki.wVk = VK_SPACE; // virtual-key code for the "space" key.
4   ip.ki.dwFlags = 0;    // 0 for key press.
5   SendInput(1, &ip, sizeof(INPUT));
6
7   // Release the "space" key.
8   ip.ki.wVk = VK_SPACE; // virtual-key code for the "space" key.
9   ip.ki.dwFlags = KEYEVENTF_KEYUP; // KEYEVENTF_KEYUP for key release.
10  SendInput(1, &ip, sizeof(INPUT));
11  Sleep(50);
12 }
13
14 void one() {
15     // Press the "1" key.
16     ip.ki.wVk = 0x31; // virtual-key code for the "1" key.
17     ip.ki.dwFlags = 0; // 0 for key press
18     SendInput(1, &ip, sizeof(INPUT));
19
20     // Release the "1" key.
21     ip.ki.wVk = 0x31; // virtual-key code for the "1" key.
22     ip.ki.dwFlags = KEYEVENTF_KEYUP; // KEYEVENTF_KEYUP for key release.
23     SendInput(1, &ip, sizeof(INPUT));
24     Sleep(50);
25 }

```

A similar method can be used to simulate mouse clicks. And example for left click follows

```

1  void leftclick() {
2      INPUT ip={0};
3      // left down
4      ip.type = INPUT_MOUSE;
5      ip.mi.dwFlags = MOUSEEVENTF_LEFTDOWN;
6      SendInput(1,&Input, sizeof(INPUT));
7
8      // left up
9      ZeroMemory(&Input, sizeof(INPUT));
10     ip.type = INPUT_MOUSE;
11     ip.mi.dwFlags = MOUSEEVENTF_LEFTUP;
12     SendInput(1,&Input, sizeof(INPUT));
13     Sleep(50);
14 }

```

5.4.1.2 Key Strokes on Linux

First the correct files must be included and an event must be setup.

```

1  extern "C" {
2      #include <xdo.h>
3  }
4
5  xdo = xdo_new(":1.0");

```

After this, functions can be setup to simulate various keys based on the specific key specifiers, two examples of such are

```

1  void space() {
2      xdo_send_keysequence_window(xdo, CURRENTWINDOW, "space", 0);
3  }
4
5  void one() {

```

```

6   xdo_send_keysequence_window(xdo, CURRENTWINDOW, "1", 0);
7   }

```

5.5 Compiler/Processor specific

The order of bytes within a binary representation of a number can be either **little endian** or **big endian**; this property is known as **endianness**. In some cases, it is important to know this. Below is a function that will return the endianness of the machine you are compiling on.

```

1  bool is_big_endian(){
2      union { uint32_t i; char c[4]; } bint = {0x01020304};
3      return bint.c[0] == 1;
4  }
5
6  #include <endian.h>    // Defines BYTE_ORDER, BIG_ENDIAN, LITTLE_ENDIAN (POSIX).
7
8  #if BYTE_ORDER == BIG_ENDIAN
9      // Use big endian code here.
10 #endif
11
12 #if BYTE_ORDER == LITTLE_ENDIAN
13     // Use little endian code here.
14 #endif

```

On Windows (MSVC), `<endian.h>` is not available; use the compiler built-ins `__BYTE_ORDER__`, `__ORDER_BIG_ENDIAN__`, and `__ORDER_LITTLE_ENDIAN__` instead (e.g., `#if __BYTE_ORDER__ == __ORDER_LITTLE_ENDIAN__`).

To define specific code to use on **Windows** vs **Linux** you can use the following

```

1  // This checks for windows or Cygwin.
2  #if defined(WIN32) || defined(_WIN32) || defined(__WIN32__) || defined(__NT__) || defined
   ↪ _WIN32 || defined _WIN64 || defined __CYGWIN__
3      // Windows only code here...
4  #elif __linux__
5      // Linux only code here...
6  #endif

```

5.5.0.1 Inspecting Dynamic Symbols

The `nm -D` command lists dynamic symbols in shared libraries or executables, useful for debugging **linker** errors. It shows symbols a library provides or requires, marked as T (defined, e.g., functions) or U (undefined, to be resolved by another library). Use with `grep` to find specific symbols.

```

1  nm -D /usr/lib64/libmyLibrary.so | grep someMethod

```

Output Example: If `U otherLibrary::someClass::someMethod` appears, the library expects `someMethod` from another library, indicating a dependency. If empty, the symbol is not referenced.

5.6 Advanced Coding Features

C++ allows you to perform **compile-time** evaluation of expressions using **constexpr** functions and variables. This feature enables you to compute values at compile-time rather than **runtime**, providing potential performance benefits and allowing for more flexible code optimization.

```

1  #include <iostream>
2
3  constexpr int fibonacci(int n) {

```

```

4     if (n <= 1) return n;
5     return fibonacci(n - 1) + fibonacci(n - 2);
6 }
7
8 int main() {
9     constexpr int fib_10 = fibonacci(10);
10    std::cout << "Fibonacci number at position 10: " << fib_10 << std::endl;
11
12    return 0;
13 }

```

5.6.0.1 user-defined literals

C++ allows you to create **user-defined literals**, which are custom suffixes that can be applied to literal values to create objects of **user-defined** types. This feature enables you to create more readable and expressive code by introducing **domain-specific literals**.

```

1 #include <iostream>
2
3 // Define a user-defined literal for converting meters to kilometers
4 constexpr long double operator"" _km(long double meters) {
5     return meters / 1000.0;
6 }
7
8 int main() {
9     // Use the user-defined literal to convert meters to kilometers
10    long double distance = 5000.0_km;
11
12    // Outputs "Distance in kilometers: 5"
13    std::cout << "Distance in kilometers: " << distance << std::endl;
14
15    return 0;
16 }

```

When using **user-defined literals**, it is up to the user to ensure that the units are consistent throughout the application. For example, the below defines literals that convert between Celsius and Fahrenheit. Each literal both tags a value with its source unit and performs the conversion, so the unit of every quantity is explicit at the point where it is written.

```

1 #include <iostream>
2
3 // Convert a Celsius value to Fahrenheit.
4 constexpr double operator"" _CtoF(long double celsius) {
5     return (celsius * 9.0 / 5.0) + 32.0;
6 }
7
8 // Convert a Fahrenheit value to Celsius.
9 constexpr double operator"" _FtoC(long double fahrenheit) {
10    return (fahrenheit - 32.0) * 5.0 / 9.0;
11 }
12
13 int main() {
14    // Convert Celsius values to Fahrenheit.
15    constexpr double freezing_point_F = 0.0_CtoF;    // 0 C -> 32 F
16    constexpr double boiling_point_F  = 100.0_CtoF;  // 100 C -> 212 F
17
18    std::cout << "Freezing point of water in Fahrenheit: " << freezing_point_F << std::
        ↪ endl;

```

```

19 // Outputs "Freezing point of water in Fahrenheit: 32"
20 std::cout << "Boiling point of water in Fahrenheit: " << boiling_point_F << std::endl
    ↪ ;
21 // Outputs "Boiling point of water in Fahrenheit: 212"
22
23 // Convert Fahrenheit values to Celsius.
24 constexpr double boiling_point_C = 212.0_FtoC; // 212 F -> 100 C
25 constexpr double body_temp_C    = 98.6_FtoC; // 98.6 F -> 37 C
26
27 std::cout << "Boiling point of water in Celsius: " << boiling_point_C << std::endl;
28 // Outputs "Boiling point of water in Celsius: 100"
29 std::cout << "Body temperature in Celsius: " << body_temp_C << std::endl;
30 // Outputs "Body temperature in Celsius: 37"
31
32 return 0;
33 }

```

5.6.0.2 templates

C++ provides a feature called "**template specialization**," which allows you to provide specialized implementations for specific **template** arguments. Template specialization is particularly useful when you need different behavior for certain template arguments while maintaining a generic implementation for others.

```

1 #include <iostream>
2
3 // Generic template
4 template <typename T>
5 struct MyTemplate {
6     void print() {
7         std::cout << "Generic template: " << typeid(T).name() << std::endl;
8     }
9 };
10
11 // Specialized template for int
12 template <>
13 struct MyTemplate<int> {
14     void print() {
15         std::cout << "Specialized template for int" << std::endl;
16     }
17 };
18
19 int main() {
20     MyTemplate<double> obj1;
21     obj1.print(); // Output: Generic template: d
22
23     MyTemplate<int> obj2;
24     obj2.print(); // Output: Specialized template for int
25
26     return 0;
27 }

```

5.6.0.3 Function Pointers

C++ allows you to define and use "**function pointers**," which are **pointers** that point to functions rather than data. Function pointers enable you to treat functions as first-class citizens, allowing you to pass functions as arguments to other functions, store them in data structures, and call them dynamically at runtime.

```
1 #include <iostream>
2
3 // Function pointer type declaration
4 typedef int (*Operation)(int, int);
5
6 // Function to add two numbers
7 int add(int a, int b) {
8     return a + b;
9 }
10
11 // Function to subtract two numbers
12 int subtract(int a, int b) {
13     return a - b;
14 }
15
16 // Function to perform an operation using function pointer
17 int perform(Operation op, int a, int b) {
18     return op(a, b);
19 }
20
21 int main() {
22     // Define function pointers
23     Operation addition = &add;
24     Operation subtraction = &subtract;
25
26     // Use function pointers to perform operations
27     std::cout << "Addition result: " << perform(addition, 5, 3) << std::endl;
28     std::cout << "Subtraction result: " << perform(subtraction, 5, 3) << std::endl;
29
30     return 0;
31 }
```

5.7 PImpl Idiom (Pointer to Implementation)

The **PImpl** (Pointer to Implementation) idiom is commonly used in C++ to hide implementation details and minimize the compilation dependencies between an interface and the implementation. It is especially useful when you want to reduce the recompilation cost when modifying the implementation of a class, as it allows you to change the private members of a class without affecting its header file.

The PImpl idiom works by moving the implementation details into a separate class, which is accessed through a pointer in the public class. The pointer is then used to forward method calls to the actual implementation.

5.7.1 Example: PImpl Idiom

Below is an example of how the PImpl idiom can be implemented:

```
1 #include <memory>
2 #include <iostream>
3
4 // Forward declaration of the implementation class.
5 class MyClassImpl;
6
7 // Public interface of MyClass, which hides the implementation details.
8 class MyClass {
9 public:
10     MyClass();
11     ~MyClass();
```



```

12     void display() const;
13
14
15 private:
16     // Pointer to the implementation (PImpl).
17     std::unique_ptr<MyClassImpl> pImpl;
18 };
19
20 // The implementation class (hidden from the public interface).
21 class MyClassImpl {
22 public:
23     void displayImpl() const {
24         std::cout << "Displaying implementation details!" << std::endl;
25     }
26 };
27
28 // Constructor initializes the pImpl pointer.
29 MyClass::MyClass() : pImpl(std::make_unique<MyClassImpl>()) {}
30
31 // Destructor ensures proper cleanup of the pImpl pointer.
32 MyClass::~MyClass() = default;
33
34 // Public method that forwards the call to the implementation.
35 void MyClass::display() const {
36     pImpl->displayImpl();
37 }
38
39 int main() {
40     MyClass obj;
41     obj.display(); // Calls the implementation's display method.
42     return 0;
43 }

```

5.7.2 Benefits of PImpl

The PImpl idiom provides several advantages:

- **Reduced Compilation Dependencies:** Changes in the implementation class do not require recompiling the public interface (header file).
- **Binary Compatibility:** The header file remains unchanged, ensuring that the ABI (Application Binary Interface) remains stable across different versions.
- **Encapsulation:** The implementation details are hidden from the user, reducing the exposure of internal data structures and methods.

5.7.3 Considerations

- **Pointer Overhead:** The use of a pointer adds an extra level of indirection, which may incur a slight performance penalty, especially when dealing with frequently accessed members.
- **Memory Management:** Proper memory management (e.g., using `std::unique_ptr`) is necessary to ensure that the implementation class is properly destroyed when the public class goes out of scope.

5.8 Tag Dispatch

Tag dispatch is a technique used in C++ to implement function overload resolution based on types that are not directly related to the arguments but serve as tags to differentiate between different implementations. It allows for more

flexible and efficient generic code by choosing the correct implementation based on type traits or other compile-time conditions.

The basic idea is to define different versions of a function or method, each specialized for a particular "tag" type. The dispatching mechanism is usually done through template specialization or **SFINAE** (Substitution Failure Is Not An Error), where the tag itself is used to determine which function version to call.

5.8.1 Example: Tag Dispatch

```

1  #include <iostream>
2  #include <type_traits>
3
4  // Tag types that will be used for dispatching.
5  struct tag_int {};
6  struct tag_float {};
7
8  // General template for the function (does nothing by default).
9  template <typename T, typename Tag>
10 void process(T value, Tag tag) {
11     std::cout << "Generic processing" << std::endl;
12 }
13
14 // Specialized version of process for integers.
15 template <>
16 void process<int, tag_int>(int value, tag_int tag) {
17     std::cout << "Processing integer: " << value << std::endl;
18 }
19
20 // Specialized version of process for floats.
21 template <>
22 void process<float, tag_float>(float value, tag_float tag) {
23     std::cout << "Processing float: " << value << std::endl;
24 }
25
26 int main() {
27     int i = 10;
28     float f = 3.14f;
29
30     process(i, tag_int{}); // Calls the integer-specific version.
31     process(f, tag_float{}); // Calls the float-specific version.
32     process(i, tag_float{}); // Calls the generic version.
33
34     return 0;
35 }

```

5.8.2 Benefits of Tag Dispatch

Tag dispatch can be particularly useful in the following cases:

- **Specialized Implementations:** Allows different code paths to be chosen based on the type of the argument in a highly flexible and extensible way.
- **Compile-Time Decisions:** The dispatch mechanism enables making decisions at compile time, resulting in more efficient and optimized code by avoiding runtime overhead.
- **Extensibility:** Adding new types or modifying the behavior for specific types requires minimal changes, making the system easy to extend.

5.8.3 Considerations

- **Complexity:** While powerful, tag dispatch can make the codebase harder to maintain due to its reliance on template metaprogramming techniques and the need to define specialized versions of functions.
- **Overhead:** In some cases, the number of specialized templates required for dispatching can lead to a larger code size, especially when dealing with many types.

5.9 Modern C++ Utility Types

Modern C++ (C++11 and later) introduces several utility types that simplify common programming tasks. These are part of the C++ Standard Library and do not require any external dependencies. They are used across a wide range of applications, from everyday code to library development, and are independent of any specific programming paradigm.

5.9.1 `std::optional`

Introduced in **C++17**, `std::optional` provides a type-safe way to represent optional values - a value that may or may not exist. This avoids the need for sentinel values such as `-1`, `nullptr`, or separate boolean flags. An `std::optional` either contains a value of the specified type or is empty (represented by `std::nullopt`).

```

1  #include <optional>
2  #include <string>
3
4  std::optional<std::string> getValue(const std::string& key) {
5      if (hasKey(key)) {
6          return getValueForKey(key);
7      }
8      return std::nullopt; // Explicitly indicates no value.
9  }
10
11 // Usage:
12 auto result = getValue("myKey");
13 if (result.has_value()) {
14     std::cout << *result << std::endl; // dereference to access the value
15 } else {
16     std::cout << "Key not found!" << std::endl;
17 }
```

`std::optional` is commonly used for return values that may not exist (e.g., lookup, parse, file read), avoiding magic sentinel values, and writing safer, more expressive code.

5.9.2 `std::make_pair` and `std::tuple`

`std::make_pair` (available since **C++11**) and `std::make_tuple` (available since **C++11**) are utility functions used to construct **pairs** and **tuples** from values. They automatically deduce the element types, making them convenient for bundling multiple values together without defining a custom struct.

```

1  #include <utility>
2  #include <tuple>
3
4  auto p = std::make_pair("name", 42);           // std::pair<const char*, int>
5  auto t = std::make_tuple(1, 2.5, "hello");     // std::tuple<int, double, const char*>
6
7  // Accessing elements of a pair
8  std::cout << p.first << ": " << p.second << std::endl; // name: 42
9
10 // Accessing elements of a tuple by index
```

```
11 std::cout << std::get<0>(t) << ", " << std::get<1>(t) << std::endl; // 1, 2.5
```

These utilities are commonly used to store multiple values together (e.g., key-value pairs), return multiple values from a function, and pass multiple arguments to a function or lambda.

5.9.3 std::apply

Introduced in **C++17**, **std::apply** calls a callable object (such as a function or **lambda**) with the elements of a **tuple** as its arguments. While **std::get<N>()** works fine when you know the tuple's structure at write time, **std::apply** is essential in generic code where the tuple's size and element types are not known until compile time. A common real-world use case is a task scheduler that stores deferred function calls as tuples of (function, arguments...) and executes them later.

```
1  #include <iostream>
2  #include <tuple>
3  #include <string>
4
5  // A generic task executor that receives a tuple of (function, args...).
6  // At this point, we have NO IDEA what's in the tuple -- how many elements,
7  // what types, etc. We just know it's a tuple that represents a callable
8  // and its arguments.
9  template<typename Tuple>
10 void executeTask(Tuple&& task) {
11     // std::apply unpacks the tuple and calls the function with its args.
12     // We couldn't write std::get<0>(task)(std::get<1>(task), std::get<2>(task)...)
13     // because we don't know how many arguments there are.
14     std::apply([&func, &args] {
15         func(std::forward<decltype(args)>(args)...);
16     }, task);
17 }
18
19 void printSum(int a, int b) {
20     std::cout << "Sum: " << a + b << std::endl;
21 }
22
23 void greet(const std::string& name, int times) {
24     for (int i = 0; i < times; i++)
25         std::cout << "Hello, " << name << "!" << std::endl;
26 }
27
28 int main() {
29     // Tasks created elsewhere -- the executor doesn't know their structure
30     auto task1 = std::make_tuple(printSum, 3, 4);
31     auto task2 = std::make_tuple(greet, std::string("World"), 2);
32
33     executeTask(task1);
34     // Output: Sum: 7
35
36     executeTask(task2);
37     // Output: Hello, World!
38     //           Hello, World!
39
40     return 0;
41 }
```

Notice that **executeTask** works for both tasks even though they have different numbers of arguments and completely different types. There is no way to write that with **std::get<N>** because you would need to hardcode the indices, and those indices are different for each task.

5.9.4 `std::tie`

Introduced in **C++11**, `std::tie` creates a tuple of references from existing variables, allowing a returned **tuple** or **pair** to be unpacked into those variables in a single assignment. It can also be combined with `std::ignore` to discard unwanted elements.

```

1  std::tuple<bool, int, std::string> getConfig() {
2      return {true, 42, "default"};
3  }
4
5  bool ok;
6  int id;
7  std::string name;
8  std::tie(ok, id, name) = getConfig();
9  // ok = true, id = 42, name = "default"
10
11 // Discard the second element
12 std::tie(ok, std::ignore, name) = getConfig();

```

If declaring new variables, C++17 **structured bindings** (`auto [ok, id, name] = getConfig();`) is generally preferred. `std::tie` remains useful when the variables already exist or when discarding elements with `std::ignore`.

5.9.5 `std::ignore`

`std::ignore` is a placeholder object that discards a value assigned to it. Its primary use is with `std::tie` to skip elements when unpacking a **tuple** or **pair**.

```

1  bool ok;
2  std::string name;
3  // Discard the int (index 1), keep the bool and string
4  std::tie(ok, std::ignore, name) = getConfig();

```

5.9.6 `std::variant`

Introduced in **C++17**, `std::variant` provides a type-safe **union**. Unlike a C-style **union**, which provides no way to know which type is currently stored, `std::variant` tracks the active alternative and provides safe access. A mismatched access throws `std::bad_variant_access`.

```

1  std::variant<int, double, std::string> value;
2
3  value = 42;
4  std::cout << std::get<int>(value) << std::endl;           // 42
5
6  value = "hello";
7  if (auto ptr = std::get_if<std::string>(&value)) {
8      std::cout << *ptr << std::endl;                       // hello
9  }
10 // std::get<int>(value); // Throws std::bad_variant_access

```

5.9.7 `std::any`

Introduced in **C++17**, `std::any` is a container that can hold a value of any type. Unlike `std::variant`, which requires enumerating all possible types upfront, `std::any` can store any type without advance declaration. The trade-off is that the exact stored type must be known to retrieve the value via `std::any_cast`, and a mismatched cast throws `std::bad_any_cast`.

```

1  std::any value;
2
3  value = 42;
4  std::cout << std::any_cast<int>(value) << std::endl;      // 42
5
6  value = std::string("hello");
7  std::cout << std::any_cast<std::string>(value) << std::endl; // hello
8
9  // std::any_cast<double>(value); // Throws std::bad_any_cast

```

When the set of possible types is known at compile time, **std::variant** is preferred because it provides compile-time type guarantees. **std::any** is useful when the type set is not known, such as in plugin systems or generic property bags.

5.10 Smart Pointers

C++11 introduced **smart pointers** as part of the `memory` header to provide automatic **memory management** and eliminate the need for manual `new` and `delete`. Smart pointers use **RAII** (Resource Acquisition Is Initialization) to ensure that memory is freed when the pointer goes out of scope, preventing memory leaks and dangling pointers.

5.10.1 std::unique_ptr

std::unique_ptr is a smart pointer that has **exclusive ownership** of a dynamically allocated object. Only one **unique_ptr** can own the object at a time. Ownership can be transferred using **std::move**, but the pointer cannot be copied.

```

1  #include <memory>
2
3  // Create a unique_ptr
4  std::unique_ptr<int> p1 = std::make_unique<int>(42);
5  // std::unique_ptr<int> p2 = p1; // ERROR: cannot copy
6
7  // Transfer ownership
8  std::unique_ptr<int> p2 = std::move(p1); // p1 is now nullptr

```

5.10.2 std::shared_ptr

std::shared_ptr is a smart pointer that allows **shared ownership** of a dynamically allocated object. Multiple **shared_ptr** objects can point to the same resource. A **reference count** tracks how many **shared_ptr** objects own the resource, and the memory is freed only when the last **shared_ptr** is destroyed or reset.

```

1  #include <memory>
2
3  // Create a shared_ptr
4  std::shared_ptr<int> p1 = std::make_shared<int>(42);
5  std::shared_ptr<int> p2 = p1; // OK: both share ownership
6
7  // Reference count is now 2
8  std::cout << p1.use_count() << std::endl; // 2
9
10 p1.reset(); // p1 gives up ownership, count drops to 1
11 // Memory is freed when p2 goes out of scope

```

5.10.3 std::weak_ptr

std::weak_ptr is a smart pointer that holds a **non-owning reference** to an object managed by a **std::shared_ptr**. It does not participate in the **reference count**, so it cannot prevent the object from being destroyed. To access the object, a **weak_ptr** must be converted to a **shared_ptr** using **lock()**, which returns a valid **shared_ptr** if the object still exists, or an empty one if it has been destroyed.

```

1  std::shared_ptr<int> shared = std::make_shared<int>(42);
2  std::weak_ptr<int> weak = shared; // Does not increase reference count
3
4  if (auto locked = weak.lock()) {
5      std::cout << *locked << std::endl; // 42
6  }
7
8  shared.reset(); // Object is destroyed, reference count hits 0
9
10 if (auto locked = weak.lock()) {
11     std::cout << *locked << std::endl; // Not reached
12 } else {
13     std::cout << "Object no longer exists" << std::endl;
14 }

```

std::weak_ptr is primarily used to break **reference cycles**. If two objects hold **shared_ptr** references to each other, neither will ever be destroyed because each keeps the other's reference count above zero. By making one direction a **weak_ptr**, the cycle is broken and both objects can be properly cleaned up.

5.10.4 std::make_unique and std::make_shared

std::make_unique (C++14) and **std::make_shared** (C++11) are factory functions that construct an object and return the appropriate smart pointer. They are preferred over directly using **new** because they are safer (no raw pointer exposed), more efficient (**make_shared** allocates the control block and object in a single allocation), and exception-safe.

```

1  auto p1 = std::make_unique<int>(42);
2  auto p2 = std::make_shared<int>(42);
3
4  // Prefer this over:
5  // std::unique_ptr<int> p1(new int(42));
6  // std::shared_ptr<int> p2(new int(42));

```

5.11 Type Aliases

C++ provides two ways to create alternative names for types. **typedef** (inherited from C) and **using** (introduced in C++11). The **using** syntax is preferred in modern C++ because it is more readable and supports **template aliasing**, which **typedef** cannot do.

```

1  // typedef syntax
2  typedef unsigned long ulong;
3
4  // using syntax (preferred)
5  using ulong = unsigned long;
6
7  // Template aliasing -- only possible with 'using'
8  template<typename T>
9  using StringMap = std::map<std::string, T>;
10
11 StringMap<int> ages; // std::map<std::string, int>

```

5.12 Range-Based For Loops

Introduced in **C++11**, **range-based for loops** provide a concise syntax for iterating over containers. C++17 extended this with **structured bindings** in the loop, allowing iteration over **maps** with unpacked key-value pairs.

```

1  std::vector<int> nums = {1, 2, 3, 4, 5};
2
3  // By value (copies each element)
4  for (int n : nums) { std::cout << n; }           // 12345
5
6  // By reference (modifies elements in place)
7  for (int& n : nums) { n *= 2; }                  // nums is now {2, 4, 6, 8, 10}
8
9  // C++17: Structured bindings with a map
10 std::map<std::string, int> ages = {"Alice", 30}, {"Bob", 25};
11 for (const auto& [name, age] : ages) {
12     std::cout << name << ": " << age << " ";
13 }
14 // Output: Alice: 30 Bob: 25

```

5.13 Lambdas

Introduced in **C++11**, a **lambda** is an anonymous function object that can be defined inline. Lambdas are useful for short functions passed to **STL algorithms**, callbacks, or any situation where defining a named function would be unnecessarily verbose. The general syntax is:

```

1  [capture](parameters) -> return_type { body }

```

The **capture** clause specifies which variables from the enclosing scope are available inside the lambda. The return type is optional and can be deduced by the compiler.

```

1  #include <algorithm>
2  #include <vector>
3
4  std::vector<int> nums = {5, 2, 8, 1, 9};
5
6  // Sort in descending order using a lambda
7  std::sort(nums.begin(), nums.end(), [](int a, int b) {
8      return a > b;
9  });
10 // nums is now {9, 8, 5, 2, 1}
11
12 // Capture by value
13 int threshold = 5;
14 auto above = [threshold](int n) { return n > threshold; };
15 // above(8) returns true, above(3) returns false
16
17 // Capture by reference
18 int counter = 0;
19 auto increment = [&counter]() { counter++; };
20 increment();
21 increment();
22 // counter is now 2

```

Common capture modes:

Capture	Meaning
[]	Capture nothing
[=]	Capture all variables by value
[&]	Capture all variables by reference
[x]	Capture x by value
[&x]	Capture x by reference
[=, &x]	Capture all by value, except x by reference
[&, x]	Capture all by reference, except x by value

5.14 Structured Bindings

Introduced in **C++17**, **structured bindings** allow decomposing an object into its constituent parts in a single declaration. This works with **tuples**, **pairs**, arrays, and simple structs with public members.

```

1 // Tuples and pairs
2 auto [success, id, message] = getConfig();
3
4 // Maps (unpacks key-value pairs)
5 std::map<std::string, int> ages = {{"Alice", 30}, {"Bob", 25}};
6 for (const auto& [name, age] : ages) {
7     std::cout << name << ": " << age << std::endl;
8 }
9
10 // Arrays
11 int arr[3] = {1, 2, 3};
12 auto [a, b, c] = arr;

```

5.15 Conditional Compilation with `std::enable_if_t`

Introduced in **C++14** as a convenience alias for `std::enable_if::type`, `std::enable_if_t` enables or disables a function template overload based on a compile-time condition. This is a cornerstone of **SFINAE** (Substitution Failure Is Not An Error), a C++ rule that allows the compiler to silently ignore a template specialization if a substitution failure occurs during overload resolution, rather than treating it as an error. By leveraging SFINAE, `std::enable_if_t` allows the compiler to select the appropriate overload from a set of candidates based on type traits or other compile-time properties.

In its general form, `std::enable_if_t<Condition, T>` evaluates `Condition` at compile time. If the condition is `true`, the expression evaluates to type `T` (which defaults to `void` if omitted), and the function template is enabled. If the condition is `false`, substitution fails and the overload is silently discarded.

```

1 #include <type_traits>
2
3 // Only enabled when T is an integral type (int, long, char, etc.)
4 template<typename T>
5 std::enable_if_t<std::is_integral_v<T>, T>
6 square(T x) {
7     return x * x;
8 }
9
10 // Only enabled when T is a floating-point type (float, double, etc.)
11 template<typename T>
12 std::enable_if_t<std::is_floating_point_v<T>, T>
13 square(T x) {
14     return x * x;
15 }

```

`std::enable_if_t` is commonly used to enable a function only when a condition is met (e.g., argument count, type trait), overload functions based on compile-time properties, and prevent misuse of generic functions.

5.16 Variadic Templates and Fold Expressions

Modern C++ (C++11 and later) provides tools for writing generic, type-safe, and efficient code. Key features include **variadic templates** and **fold expressions**. These are typically used in library development, metaprogramming, and systems code where compile-time flexibility and zero-runtime-overhead abstractions are essential.

5.16.1 Variadic Templates

Introduced in **C++11**, **variadic templates** allow templates to accept any number of type or value parameters. This is accomplished through the use of **parameter packs**, which can represent zero or more types or values. A type parameter pack is declared with `typename... Args`, while a value parameter pack is declared with `Args... args`. The `sizeof...` operator returns the number of elements in a parameter pack at compile time.

```
1 template<typename... Args>
2 void func(Args... args);
3
4 // sizeof...(Args) returns the number of types in the pack
5 // sizeof...(args) returns the number of values in the pack
```

Variadic templates are commonly used for functions that take any number of arguments (e.g., logging, formatting, property copying), compile-time checks with `static_assert` and `sizeof...`, and generic containers or utilities that operate on an arbitrary number of types.

A common metaprogramming pattern using variadic templates is the **type list** - a struct that stores a list of types in its template parameters without holding any data. The type information is accessed at compile time through **partial template specialization**, where utility structs are specialized on `TypeList<Types...>` and peel types off the pack one at a time through recursion. The following example demonstrates a type list with utilities to query its size and retrieve a type at a specific index.

```
1 #include <iostream>
2 #include <typeinfo>
3
4 // The type list itself -> just an empty shell that holds type info.
5 // in its template parameters, not in any data member.
6 template<typename... Types>
7 struct TypeList {};
8
9 // Compile-time utilities that operate on a TypeList.
10
11 // 1. Get the number of types in the list:
12 template<typename List>
13 struct Size;
14
15 template<typename... Types>
16 struct Size<TypeList<Types...>> {
17     static constexpr size_t value = sizeof...(Types);
18 };
19
20 // 2. Get the Nth type in the list:
21 template<size_t N, typename List>
22 struct TypeAt;
23
24 // Base case: index 0 returns the first type (Head)
25 template<typename Head, typename... Tail>
26 struct TypeAt<0, TypeList<Head, Tail...>> {
```

```

27     using type = Head;
28 };
29
30 // Recursive case: decrement N and recurse into the remaining types (Tail).
31 template<size_t N, typename Head, typename... Tail>
32 struct TypeAt<N, TypeList<Head, Tail...>> {
33     using type = typename TypeAt<N - 1, TypeList<Tail...>>::type;
34 };
35
36 // Using it:
37 using MyTypes = TypeList<int, double, std::string>;
38
39 int main() {
40     std::cout << "Number of types: " << Size<MyTypes>::value << std::endl;
41     // Output: Number of types: 3
42
43     std::cout << "Type at index 1: " << typeid(TypeAt<1, MyTypes>::type).name() << std::
44         ↪ endl;
45     // Output: Type at index 1: d (gcc's mangled name for double)
46
47     return 0;
48 }

```

This pattern is used in real libraries. For example, `std::variant` and `std::tuple` internally use similar techniques to store a list of types and generate the appropriate code for each one.

5.16.2 Parameter Pack Expansion

Parameter pack expansion expands a parameter pack into multiple arguments using the `...` syntax. This is fundamental to working with variadic templates, as it allows the pack to be “unpacked” into individual arguments when passed to functions, constructors, or other templates.

```

1  template<typename... Args>
2  auto makeTuple(Args... args) {
3      return std::make_tuple(args...); // args... expands the pack
4  }

```

Parameter pack expansion is commonly used to build **tuples**, arrays, or containers from a pack, pass arguments to functions or constructors, and generate multiple calls or assignments.

5.16.3 Fold Expressions

Introduced in **C++17**, **fold expressions** provide a concise way to apply a binary operator across all elements of a parameter pack at compile time. Prior to C++17, achieving the same result required recursive template instantiation. Fold expressions eliminate this boilerplate and are evaluated entirely at compile time with no runtime cost.

```

1  // Unary right fold: (pack op ... )
2  (args && ...)           // true if all args are true
3
4  // Unary left fold: (... op pack)
5  (... + args)           // sum of all args
6
7  // Binary fold: (init op ... op pack)
8  (0 + ... + args)       // sum with explicit initial value

```

Fold expressions are commonly used to check if all elements satisfy a condition (e.g., `&&` fold), sum or combine values (e.g., `+` fold), and call a function on each element of a pack (e.g., with the comma operator). The following example shows a function that returns `true` only if all boolean arguments are `true`.

```

1  template<typename... Args>
2  bool allTrue(Args... args) {
3      return (args && ...); // unary right fold over &&
4  }

```

5.16.4 Perfect Forwarding

Introduced in C++11, **perfect forwarding** allows a function template to forward its arguments to another function while preserving the argument's value category (i.e., whether it is an **lvalue** or **rvalue**). This is achieved through **forwarding references** (also known as **universal references**), which are declared using **T&&** in a deduced context, combined with **std::forward** to cast the argument appropriately.

```

1  template<typename T>
2  void forwardArg(T&& arg) {
3      someFunction(std::forward<T>(arg)); // preserves lvalue/rvalue-ness
4  }

```

Perfect forwarding is essential for generic functions that accept any argument type, factory functions and wrappers that construct objects in-place, and avoiding unnecessary copies in template code. When combined with variadic templates, it enables functions to forward any number of arguments perfectly.

```

1  template<typename... Args>
2  void emplace(Args&&... args) {
3      // Forwards all arguments, preserving their value categories
4      construct(std::forward<Args>(args)...);
5  }

```

5.16.5 Summary of Variadic Template Features

5.1: Variadic Template Features

Feature	Standard	Primary Use
Variadic Templates	C++11	Accept any number of types or values
Parameter Pack Expansion	C++11	Expand pack into multiple arguments
Fold Expressions	C++17	Apply operator across all elements of a pack
Perfect Forwarding (&&...)	C++11	Forward arguments without copying

These features are essential for writing modern, generic, and efficient C++ code. They allow developers to write type-safe code that catches errors at compile time, avoid runtime overhead by processing data at compile time, write reusable code that works with any number of types or values, and create library-level abstractions that are both powerful and easy to use. They are used extensively throughout the C++ Standard Library (e.g., **std::tuple**, **std::variant**, **std::any**), as well as in game engines, messaging systems, serialization libraries, high-performance systems, and embedded code.

5.17 Lvalues and Rvalues

In C++, every expression has both a type and a **value category**. The two most important value categories are **lvalue** and **rvalue**. An **lvalue** (“locator value”) is an expression that refers to a persistent object in memory - it has an address and can be assigned to. An **rvalue** (“read value”) is a temporary value that does not persist beyond the expression that creates it - it can be read but not assigned to.

```

1  int x = 10;
2  int y = 20;

```

```

3
4 x = y;           // x is an lvalue (has an address, can be assigned to).
5                 // y is also an lvalue (used as the source of the value).
6
7 x = x + y;       // x on the left is an lvalue.
8                 // (x + y) is an rvalue (a temporary result, no address).
9
10 int& ref = x;     // OK: An lvalue reference can bind to an lvalue.
11 // int& bad = 5; // ERROR: An lvalue reference cannot bind to an rvalue.
12
13 int&& rref = 5;    // OK: An rvalue reference can bind to an rvalue.
14 rref = 10;        // OK: rref is now an lvalue (it has a name and address).

```

Understanding the distinction is essential for **move semantics** and **perfect forwarding**. When an rvalue is assigned to an **rvalue reference** (a kind of **reference** declared with `&&`), it signals that the value is temporary and its resources can be “stolen” (moved) rather than copied.

5.18 Move Semantics

Introduced in **C++11**, **move semantics** allow resources (e.g., heap memory, file handles) to be transferred from one object to another rather than copied. This avoids expensive deep copies of temporary objects. A **move constructor** and **move assignment operator** are invoked when the source is an **rvalue**, allowing the destination to “steal” the internal pointer and leave the source in a valid but empty state. The `std::move` function casts an lvalue to an rvalue reference, enabling the move.

```

1 std::string a = "Hello, World!";
2
3 // Without std::move: Copies the string data (expensive).
4 std::string b = a;
5
6 // With std::move: Transfers the internal buffer (cheap).
7 // 'a' is left in a valid but unspecified state (usually empty).
8 std::string c = std::move(a);

```

5.19 Virtual Functions and the Vtable

When a class contains **virtual functions**, the compiler implements **dynamic dispatch** (runtime polymorphism) using a hidden data structure called a **vtable** (virtual method table). Each class with virtual functions gets a single vtable containing pointers to the virtual function implementations for that class. Every instance of that class secretly holds a hidden pointer (the **vptr**) to its class’s vtable.

```

1 class Base {
2 public:
3     virtual void doWork() { std::cout << "Base" << std::endl; }
4 };
5
6 class Derived : public Base {
7 public:
8     void doWork() override { std::cout << "Derived" << std::endl; }
9 };
10
11 int main() {
12     Base* obj = new Derived();
13     // At runtime, obj->vptr points to Derived's vtable.
14     // The vtable points to Derived::doWork.
15     obj->doWork(); // Output: Derived.

```

```

16     delete obj;
17     return 0;
18 }

```

The key takeaway is that virtual calls incur a small runtime overhead (two pointer dereferences: one to get the `vp`tr, one to get the function pointer from the vtable) compared to non-virtual calls. A virtual call also prevents the compiler from inlining the function. The **override** keyword (C++11) is recommended to ensure the function actually overrides a base class virtual function, producing a compiler error if the signature does not match.

5.20 RAII (Resource Acquisition Is Initialization)

RAII is a core C++ programming idiom where a resource's **lifetime** is tied to the lifetime of an object. The resource (memory, file handle, socket, mutex) is acquired in the object's constructor and released in its destructor. Because C++ guarantees that destructors run when an object goes out of **scope** (even via exceptions), RAII guarantees leak-free resource management without manual cleanup.

```

1  #include <fstream>
2  #include <mutex>
3
4  void processData() {
5      // File is opened in the constructor.
6      std::ifstream file("data.txt");
7      if (!file) return;
8
9      // Mutex is locked in the constructor.
10     std::lock_guard<std::mutex> lock(my_mutex);
11
12     // ... do work ...
13
14     // No manual cleanup needed!
15     // 'lock' unlocks the mutex in its destructor when it goes out of scope.
16     // 'file' closes the file handle in its destructor.
17 } // Destructors run here in reverse order of construction.

```

Smart pointers (see Section on Smart Pointers) are themselves an application of RAII to dynamic memory.

5.21 SFINAE (Substitution Failure Is Not An Error)

SFINAE is a C++ language rule that applies during **overload resolution** for templates. If substituting a template parameter fails (e.g., a type does not have a required member), the compiler does not issue an error. Instead, it silently removes that template from the candidate list and continues checking other overloads. It is the mechanism that makes `std::enable_if_t` work.

```

1  #include <iostream>
2
3  // This overload is removed via SFINAE if T does not have a 'size()' method.
4  template<typename T>
5  auto getSize(const T& val) -> decltype(val.size()) {
6      return val.size(); // Valid for std::vector, std::string, etc.
7  }
8
9  // This fallback overload is used when val.size() fails substitution.
10 template<typename T>
11 auto getSize(const T& val) -> decltype(sizeof(T)) {
12     return sizeof(val); // Falls back to byte size for int, double, etc.
13 }
14

```

```

15 int main() {
16     std::string s = "hello";
17     std::cout << getSize(s) << std::endl;    // Output: 5 (string length).
18
19     int x = 42;
20     std::cout << getSize(x) << std::endl;    // Output: 4 (sizeof int).
21     return 0;
22 }

```

5.22 auto and decltype

Introduced in **C++11**, **auto** lets the compiler deduce a variable's type from its initializer. **decltype** inspects an expression's type at compile time. Both reduce verbosity, especially with complex iterator or template types, and keep code maintainable when types change.

```

1  std::map<std::string, std::vector<int>>> data;
2
3  // auto deduces the complex iterator type.
4  for (auto it = data.begin(); it != data.end(); ++it) {}
5
6  // decltype extracts the type of an expression.
7  int x = 5;
8  decltype(x) y = 10; // y is deduced as int.

```

5.23 constexpr and consteval

constexpr (C++11) asserts that a function or variable can be evaluated at **compile time**. If the inputs are constant expressions, the compiler computes the result during compilation rather than at runtime, inlining the final value directly into the binary. **constexpr** (C++20) is stricter: it forces the function to *only* be evaluated at compile time. If called with runtime values, it produces a compiler error.

```

1  // Can run at compile time or runtime.
2  constexpr int square(int x) { return x * x; }
3
4  // MUST run at compile time only.
5  consteval int cube(int x) { return x * x * x; }
6
7  int main() {
8      int runtime_val = 5;
9
10     int a = square(runtime_val); // OK: Evaluated at runtime.
11     int b = square(5);           // OK: Evaluated at compile time.
12
13     int c = cube(5);             // OK: Evaluated at compile time.
14     // int d = cube(runtime_val); // ERROR: consteval requires compile-time input.
15     return 0;
16 }

```

5.24 nullptr

Introduced in **C++11**, **nullptr** is a strongly-typed null pointer constant. It replaces the traditional **NULL** (which is defined as 0 or `((void*)0)`). Using **nullptr** prevents ambiguity in function overload resolution, where **NULL** or 0 could be misinterpreted as an integer.

```
1 void process(int x)    { std::cout << "Integer" << std::endl; }
2 void process(int* p)  { std::cout << "Pointer" << std::endl; }
3
4 int main() {
5     process(NULL);      // Ambiguous: Might call process(int).
6     process(nullptr);   // Unambiguous: Calls process(int*).
7     return 0;
8 }
```

5.25 User-Defined Type Conversions

C++ allows a class to define how it implicitly or explicitly converts to other types. An **implicit conversion** is defined by a non-explicit constructor taking a single argument, or a **conversion operator**. An **explicit conversion** (C++11) requires an explicit cast or construction, preventing accidental silent conversions.

```
1 class Meter {
2     double value;
3 public:
4     // Allow implicit conversion from double to Meter.
5     Meter(double v) : value(v) {}
6
7     // Allow explicit conversion from Meter to double.
8     explicit operator double() const { return value; }
9 };
10
11 int main() {
12     Meter m = 5.5;           // Implicit conversion: double to Meter.
13     // double d = m;         // ERROR: 'operator double' is explicit.
14     double d = static_cast<double>(m); // OK: Explicit cast required.
15     return 0;
16 }
```


Make/CMake

CMake is an open-source, cross-platform build system generator designed to facilitate the build process for software projects. Developed by Kitware in 2000, CMake simplifies the process of building, testing, and packaging software by providing a unified configuration language that abstracts away platform-specific details. CMake generates native build scripts for various platforms and compilers, including Unix Makefiles, Visual Studio solutions, and Xcode projects, allowing developers to maintain a single set of build instructions for multiple environments. With its modular and extensible architecture, CMake supports a wide range of project structures and dependencies, making it suitable for projects of all sizes and complexities. Its widespread adoption and active community support have established CMake as a standard build tool in the software development industry.

6.1 CMake

A poorly constructed and hard to follow yet fairly comprehensive example of how to use CMake and CMakeLists.txt files can be found at the following link (which is where much of the information in this section is derived from).

URL

<https://cmake.org/cmake/help/latest/guide/tutorial/index.html>

A basic project with CMake will contain an executable built from source code. To use CMake this project must contain a **CMakeLists.txt** file containing the following.

```
1 cmake_minimum_required( VERSION 3.14 )
2
3 # Set the project name and version.
4 project( ProjectName VERSION 1.001)
5
6 # Add an executable.
7 add_executable( Main main.cpp )
8
9 # Create a binary tree to search for include files.
10 target_include_directories( Tutorial PUBLIC "${PROJECT_BINARY_DIR}" )
```

To define and enable support for a specific **C++ standard**, you can use the following.

```
1 # Specify the C++ standard to use.
2 set(CMAKE_CXX_STANDARD 11)
3 set(CMAKE_CXX_STANDARD_REQUIRED True)
```

To add a **library** in a subdirectory and use that library in the main level, you must define where the library is in the main level CMakeLists.txt file.

```
1 # Add the LibraryName library.
2 add_subdirectory( LibraryDirectory )
3
4 # Links this Main file to the desired target library.
5 target_link_libraries( Main PUBLIC LibraryName )
6
7 # Add the binary tree to the search path for include files so that we will find
8   ↳ LibraryName.h
9 target_include_directories( Main PUBLIC
10     "${PROJECT_BINARY_DIR}"
11     "${PROJECT_SOURCE_DIR}/LibraryDirectory"
12 )
```

In the directory containing the library, a CMakeLists.txt file must also exist and contain the following.

```

1 # Defines the file as a library.
2 add_library( LibraryName LibraryName.cpp)

```

To create **optional arguments** that can be turned on and off, one can use the **option** command.

```

1 # Creates a variable USE_MYLIBRARY and set it to on.
2 option( USE_MYLIBRARY "Use my library with this project" ON )

```

Variables like the above can be used as follows.

```

1 if( USE_MYLIBRARY )
2     add_subdirectory(MathFunctions)
3     list(APPEND EXTRA_LIBS LibraryName)
4     list(APPEND EXTRA_INCLUDES "${PROJECT_SOURCE_DIR}/LibraryDirectory")
5 endif()
6
7 # add the executable
8 add_executable( Main main.cpp )
9
10 target_link_libraries( Main PUBLIC ${EXTRA_LIBS} )
11
12 # Add the binary tree to the search path for include files.
13 target_include_directories( Main PUBLIC
14     "${PROJECT_BINARY_DIR}"
15     ${EXTRA_INCLUDES}
16 )

```

The use of variables defined in CMake can be defined in the source code using

```

1 #cmakedefine USE_MYLIBRARY

```

You can also define a variable for use within a C++ file as follows

```

1 # CMakeList code here:
2 project(MIA VERSION 0.300)
3 add_definitions ( -DMIVERSION="\${VERSION}\" )
4
5 // C++ code then looks like this:
6 #ifndef MIAVERSION
7     #define MIA_VERSION_VALUE MIAVERSION
8 #else
9     #define MIA_VERSION_VALUE "Unknown"
10 #endif
11 std::string Configurator::ProgramVariables::MIA_VERSION = MIA_VERSION_VALUE;

```

To prevent needing a full relative file path in a cpp file include, you can use include the directory within the CMakeList file from that directory.

```

1 # CMake code:
2 include_directories(some/relative/path)
3
4 // C++ code
5 // #include "some/relative/path/file.hpp" // No longer needed
6 #include "file.hpp" // This replaces it

```

Git

Git is a distributed version control system designed for software development, enabling programmers to manage and track changes to their code base efficiently. Utilizing a decentralized architecture, Git allows developers to work collaboratively on projects, facilitating concurrent development and seamless merging of code changes. With features such as branching, tagging, and commit history, Git empowers programmers to maintain code integrity, experiment with new features, and revert changes when necessary. Its robust branching model facilitates the creation of feature branches for code development, enabling developers to isolate changes and merge them back into the main code base effortlessly. Overall, Git serves as an indispensable tool for programmers, offering a reliable and scalable solution for version control in software development projects.

7.1: Git Resources

Various git resources exist for use of or with git (an open source, distributed version-control system).

Description	Source
Main git website	https://git-scm.com/
git book	https://git-scm.com/book/en/v2
git reference	https://git-scm.com/docs/

7.0.1 Basic Git

To turn an existing directory into a git repository you can use the git **init** command,

```
1 git init    # Makes the current directory a git repository.
```

Git contains some basic **configuration** that can be set for all the local repositories.

```
1 # Sets the name you want attached to your commit transactions.
2 git config --global user.name "[name]"
3
4 # Sets the email you want attached to your commit transactions.
5 git config --global user.email "[email address]"
6
7 # Enables helpful colorization of command line output.
8 git config --global color.ui auto
```

Some basic commands used by git include cloning repositories with the **clone** (a local version of a repository, including all commits and branches) command, checking the **log** of previous changes, and checking the **status** of current files.

```
1 git clone https://github.com/torodean/Antonius-Handbook-II.git
2
3 git log          # Prints a list of the commits and their messages.
4 git log --stat   # Shows individual file changes along with the log.
5
6 git status       # Checks which state current files are in.
```

To **synchronize** your local repository with the remote repository.

```
1 git fetch    # Downloads all history from the remote tracking branches.
2 git merge    # Combines remote tracking branch into current local branch.
3 git pull     # A combination of git fetch and git merge.
```

To record changes to a git repository, you will primarily use the **add**, **commit** (a Git object, a snapshot of your entire repository compressed into a SHA), **diff**, and **push** commands. The **add** command can be thought of to “add precisely this content to the next commit”.

```

1 git add -A           # Stages all files to be committed.
2 git add textFile.txt # Stages a text file named 'textFile'.
3 git rm textFile.txt  # Removes a text file from staging.
4
5 git diff             # Shows unstaged changes.
6 git diff --cached    # Shows staged changes.
7
8 git commit           # Commits changes and asks for commit message.
9 git commit -m "text" # Commits changes with a string as the commit message.
10 git commit -v        # includes the diff output into the commit.
11
12 git push             # Updates remote references using local references.

```

The **branch** (a lightweight movable pointer to a commit) command is used to create a new branch. The **checkout** command is used to change the working branch.

```

1 git branch           # Lists all branches.
2 git branch --merged  # See merged to current branches.
3 git branch -v        # See last commit on each branch.
4
5 git branch issue087  # Creates a branch issue087.
6 git checkout issue087 # Changes to the branch issue087.
7 git branch -d issue087 # Deletes the branch issue087.
8
9 git checkout -b issue087 # Does both of the above commands in one line.

```

The **merge** command is used to combine multiple branches after work on them is finished. To assist with merge conflicts, you can use **mergetool**.

```

1 git checkout master # Changes to the master branch.
2 git merge issue087  # Attempts to merge master and issue087.
3
4 git mergetool       # Starts mergetool to assist with merge conflicts.

```

Using the **fork** command, one can create a copy of a repository owned by a different user.

```

1 git fork [URL]

```

Sometimes a file or type of file(s) are desired to be ignored by git and not push to the repository. This can be done by creating a special file named **.gitignore**.

```

1 # Example .gitignore file. This file tells git to ignore the following types of files.
2 *.aux
3 *.log
4 *.synctex.gz
5 *.toc
6 *.o

```

7.0.2 Advanced Git

A useful tool is **stash** (code must be staged to be stashed), which lets you save code without making a commit [6][7].

```

1 git stash           # Makes a temporary local save of your repository.
2 git stash list      # Show's a list of stashes that have been made.
3 git stash apply     # Reapplies the content of a stash.
4 git stash branch    # Carry over stashed commits to new branch.

```

```

5 git drop          # Used to remove stashes individually.
6 git stash clear   # Used to remove all stashes.
7
8 git checkout .    # Resets all uncommitted code.

```

The **reset** tool is used for accidental commits or reversing commits [6][7].

```

1 git reset          # Lets you modify your repository before doing a commit.
2 git reset --soft HEAD~NUM. # Resets the most recent $NUM of commits.
3 git reset --hard HEAD~NUM # Erases your last $NUM of commits.

```

The **bisect** tool will present you with the details of a commit when compared with another. By referencing a good commit and a bad commit, git will traverse between the two and ask you which ones are good and which ones are bad and then you can display the differences after the process is finished [6][7].

```

1 git bisect          # Allows you to hunt for bad commits.
2 git bisect start    # Tells git that there is a bad commit.
3 git bisect bad      # Tells git which commit is bad.
4 git bisect good     # Tells git which commit is good.
5 git show            # Show the commit to indentify the issue.

```

The **rebase** tool is used to combine commits. The rebase tool can be dangerous as it could potentially permanently delete files. It is recommended to view the documentation before using it [6][7].

```

1 git rebase          # Allows for applying changes from one branch onto another.

```

To view and change the url that the git repository is using to update, you can use the git **remote** command.

```

1 git remote -v       # Lists the remote url of the repository.
2 git remote set-url origin <NEW-URL> # Change the url of the repo.

```

If you've already pushed a commit to a remote repository, you can still modify the most recent **commit message** (assuming the branch you are pushing to is not protected). To do so, you use the **amend** flag with git **commit**. The **force-with-lease** flag checks if the remote branch has updates that do not exist locally.

```

1 # Update the commit message.
2 git commit --amend -m "New commit message"
3
4 # Force push to the branch.
5 git push --force-with-lease origin your-branch-name

```

On **Windows**, you may see a large number of files listed as **modified** when running a git **status**. This could be caused by the way **Windows** machines automatically modify and change **file endings**. To fix this in git, you can use the **core.autocrlf** setting. Setting it to true means that Git will automatically convert **line endings** to the appropriate format for the operating system you're working on when you check out files.

```

1 git config --global core.autocrlf true

```

7.0.3 Git Submodules

Git submodules are a feature in Git that allows developers to include one Git repository as a subdirectory within another Git repository. This enables the management of dependencies and external libraries within a project, allowing for modular and organized codebases. With Git submodules, developers can easily incorporate external code repositories into their projects while maintaining version control and tracking changes across multiple repositories. This provides a convenient way to reuse code, collaborate with external teams, and manage complex project dependencies effectively. However, it's important to note that working with Git submodules requires understanding of their workflow and potential complexities, making them a powerful but nuanced tool for managing project dependencies in software development.

For a project containing git **submodules**. When cloned (by default), the submodule repo's and containing files will not be cloned. Only a reference to the submodule commit will be cloned. To clone the submodule's files along with the repo, use

```
1 git clone --recurse-submodules <project-path>
```

If you have cloned a repository and would like to update submodule files after the fact, you can use

```
1 git submodule update --init --recursive
```

In a repository containing a submodule, the submodule is always pointing to a specific commit for that submodule. This commit may not be the latest of the submodule. To update all submodules to their latest commits, you can use the **remote flag**. After doing this to update submodules, changes will appear in the top level git repository - as the submodule commit pointers have changed. These changes will need committed to the git repo.

```
1 git submodule update --init --recursive --remote
```

By default, each submodule will be in a detached **HEAD** state. To commit changes to a submodule, you must first change to a branch and then commit the changes. The top level repository contains a **.gitmodules** file that contains information on which branch the git repo will use for each submodule

```
1 # Example .gitmodules file
2 [submodule "submodule_name"]
3     path = submodule_path
4     url = submodule_url
5     branch = submodule_branch
```

A Git command that allows you to run a given shell command in each submodule of your repository is the **foreach** command. This command is particularly useful when you're working with a repository that contains multiple submodules and you need to perform an operation across all of them.

```
1 git submodule foreach <command>
```

This can be used with any other git commands to perform the action on each submodule recursively.

```
1 # Pull changes for each submodule from the main branch.
2 git submodule foreach git pull origin main
3
4 # Get the status of each submodule.
5 git submodule foreach git status
```

7.1 Github.io

GitHub Pages is a static site hosting service provided by **GitHub** that allows users to host websites directly from their GitHub repositories. Users can create and publish websites for projects, documentation, personal portfolios, or blogs by simply committing and pushing HTML, CSS, and JavaScript files to a designated repository branch. GitHub Pages supports custom domains, HTTPS encryption, and various themes, making it a convenient and accessible platform for hosting static websites. Additionally, it integrates seamlessly with GitHub's version control system, enabling easy collaboration and versioning of website content.

URL

For a free tutorial on creating a github.io website, you can follow the following link:
<https://gitwebsitetutorial.github.io/>

7.2 Quality Of Life Hacks And Scripts

7.2.1 Repository Updating

It is often the case that there are many git repositories stored in the same directory. In such an instance, it is also the case that the user may want to update all of them. This is a simple bash script to update all of the git repositories within a directory. This can be appropriately modified for other use cases and can also be modified to update submodules if desired.

```
1  #!/bin/bash
2
3  # Iterate through each directory in the current location
4  for dir in */; do
5      # Check if it's a directory
6      if [ -d "$dir" ]; then
7          # Enter the directory
8          cd "$dir" || continue
9
10         # Check if it's a Git repository
11         if [ -d ".git" ]; then
12             echo "...Updating repository for $dir"
13             # Update the repository
14             git pull
15             git status
16             echo "~~~~~"
17         else
18             echo "Skipping $dir - Not a Git repository"
19         fi
20
21         # Go back to the original directory
22         cd - >/dev/null || exit
23     fi
24 done
```

L^AT_EX

LaTeX is a typesetting system widely used in technical and scientific documentation, renowned for its exceptional typesetting quality and support for complex mathematical equations and symbols. Programmers often leverage LaTeX to produce high-quality documents, reports, articles, and presentations with precise formatting and layout control. Its markup language allows for the creation of structured documents using plain text, enabling version control and collaboration using tools like Git. LaTeX's extensive package ecosystem offers additional functionality for generating bibliographies, tables, graphics, and custom document layouts, making it a preferred choice for programmers seeking precise and professional document production capabilities.

URL

For a repository of pre-made LaTeX templates, see:
<https://github.com/torodean/Antonius-Templates>

8.1 Hello World

To use this LaTeX document, you need to have a LaTeX distribution installed on your system, such as **TeX Live** or **MiKTeX**. Save the code in a file with a .tex extension, for example, hello_world.tex. Then, compile the document using a LaTeX compiler. This will generate a PDF output containing the "Hello, World!" message. You can view the PDF using any PDF viewer application. This example serves as a basic introduction to creating LaTeX documents and demonstrates the simplicity of LaTeX syntax for generating formatted text.

```
1 % Defines the type of document (article, report, book, etc.)
2 \documentclass{article}
3
4 \begin{document}      % Begins the document environment
5 Hello, World!         % Displays the text "Hello, World!"
6 \end{document}       % Ends the document environment
```

8.2 L^AT_EX Basics

To add **comments** in LaTeX, the percent sign is used on the line of the comment.

```
1 % This is a LaTeX comment
```

To include a **package** in LaTeX, the `\usepackage{}` command will import code from a LaTeX package. the package must be installed on the machine you are trying to import from.

```
1 \usepackage{multicol}
```

8.2.1 LaTeX Titles

In LaTeX, you can add **titles**, **subtitles**, and **authors** to your documents using specific commands. These titles are typically placed at the beginning of the document and help provide context and information about the content of the document. To add a title to your document, you can use the `\title{}` command. Inside the curly braces, you specify the title of your document. For example:

```
1 \title{My LaTeX Document}
```

If you want to include a **subtitle**, you can use the `\subtitle{}` command. For example:

```
1 \subtitle{A Beginner's Guide}
```


To specify the **author(s)** of the document, you can use the `\author{}` command. Multiple authors can be separated by the `\and` command. For example:

```
1 \author{John Doe \and Jane Smith}
```

Once you have defined the title, subtitle, and author(s), you can use the `\maketitle` command to generate the title page. This command should be placed after the `\begin{document}` command. The `\maketitle` command will format and display the title, subtitle, and author(s) according to the document class and style settings.

```
1 \maketitle
```

8.3 LaTeX elements

8.3.0.1 Quotations

For a simple, fancy, and nice looking quotation box, the following environment can be defined.

```
1 % Required packages.
2 \usepackage{fontawesome5}
3 \usepackage{tcolorbox}
4
5 % Create a custom color.
6 \definecolor{lightpurple}{RGB}{220,180,240}
7
8 % Create a custom quotation box environment.
9 \newenvironment{quotationbox}{
10     \begin{tcolorbox}[
11         colback=lightpurple!10, % Very light purple background color of the box
12         colframe=lightpurple!50, % Border color of the box
13         arc=0pt, % Adjust the corner radius of the box
14         boxrule=1pt, % Border thickness
15         title={\faQuoteLeft\ Quotation}, % The quotation icon and title
16         fonttitle=\bfseries, % Font style for the title
17         coltitle=blue!50!black, % Color for the title text
18         colbacktitle=yellow!20, % Background color for the title
19         attach title to upper={\par},
20         boxsep=0pt, % Adjust the space between the content and box
21     ]
22 }{
23     \end{tcolorbox}
24 }
```

8.3.0.2 URLs

For a simple, fancy, and nice looking url box, the following environment can be defined.

```
1 % Required packages.
2 \usepackage{fontawesome5}
3 \usepackage{tcolorbox}
4
5 % Create a custom url box environment.
6 \newenvironment{urlbox}{
7     \begin{tcolorbox}[
8         colback=blue!10, % Very light green background color of the box
9         colframe=blue!50, % Border color of the box
10         arc=0pt, % Adjust the corner radius of the box
11         boxrule=1pt, % Border thickness
12         title={URL}, % The lightbulb icon and title
```

```
13     fonttitle=\bfseries,      % Font style for the title
14     coltitle=blue!50!black,   % Color for the title text
15     colbacktitle=yellow!20,   % Background color for the title
16     attach title to upper={\par},
17     boxsep=0pt,              % Adjust the space between the content and box
18   ]
19   }{
20   \end{tcolorbox}
21 }
```

For automatic **hyperlinking**, the `hyperref` package can be used.

```
1 \usepackage[unicode]{hyperref}
2
3 \url{www.duckduckgo.com}
```

C#

C# is a powerful, object-oriented programming language developed by Microsoft as part of the .NET initiative in the early 2000s. Designed for building robust, scalable applications, C# combines the productivity of modern, high-level languages with the performance and control of low-level languages. With its elegant syntax, rich standard library, and seamless integration with the .NET framework, C# enables developers to create a wide range of applications, from web and desktop applications to mobile and gaming platforms. Key features of C# include automatic memory management, type safety, and support for modern programming paradigms such as asynchronous programming and LINQ (Language-Integrated Query). Backed by a vibrant developer community and extensive documentation, C# continues to be a popular choice for building enterprise-grade software solutions across various industries.

The language C# is very similar to C++ and Java. All programming snippets listed in this section were tested and from a program created in Visual Studio 2013. Many of the functions written in this section depend on the Windows .Net application framework and may not function without that.

9.1 Useful Application Functions

Exit a program.

```

1 //These are needed at the beginning of the file.
2 using System;
3 using System.Windows.Forms;
4
5 //This exits the program
6 public static void exitLOLA(){
7     if (System.Windows.Forms.Application.MessageLoop){
8         System.Windows.Forms.Application.Exit();    // WinForms app
9     } else {
10         System.Environment.Exit(1);    // Console app
11     }
12 }
```

The following will return the current Epoch time in seconds. This is useful for version control, random number generation, and more. Also, a demonstration of how to set the current date as a string.

```

1 //Sets the current date as a string.
2 private static string today = System.DateTime.Today.ToString("d");
3
4 //Returns the current epoch time in seconds (time passed since January 1, 1970).
5 public static long getEpochTime(){
6     var epoch = (DateTime.UtcNow - new DateTime(1970, 1, 1, 0, 0, 0, DateTimeKind.Utc)).
7         ↪ TotalSeconds;
8     return (long)epoch;
9 }
```

The following can be used to increase the size of a terminal window for a terminal application made in Visual Studio.

```

1 //Doubles the length of the output terminal window if the resolution on the computer
2 ↪ permits it.
3 //Otherwise leaves it as the default.
4 public static void setScreenSize_double(){
5     //determines the screen resolution to then determine the size of the output window.
6     int origWidth = Console.WindowWidth;
7     int origHeight = Console.WindowHeight;
8     int height;
9     int screenHeight = Screen.PrimaryScreen.Bounds.Height;
10
11     if (screenHeight < 1080){
```

```

11         height = origHeight;
12     } else {
13         height = origHeight * 2;
14     }
15
16     //int height = origHeight;
17     Console.SetWindowSize(origWidth, height);
18 }

```

Get the directory path that the executable file is located in.

```

1 //returns the path that the program executable file is in.
2 public static string getProgramPath(){
3     string path = System.IO.Path.GetDirectoryName(Assembly.GetEntryAssembly().Location);
4     return path;
5 }

```

9.2 Getting Windows System Information

Return the system name

```

1 //Returns the user defined system name.
2 public static string getSystemName() {
3     systemName = Environment.MachineName;
4     return systemName;
5 }

```

Determines and returns whether a processor is 32 or 64 bits and returns the number of bits as an int.

```

1 //Returns twwhether the processor is 32 or 64-bit.
2 public static int getBits(){
3     bool bitOS = Environment.Is64BitOperatingSystem;
4     if (bitOS){
5         bits = 64;
6         return bits;
7     } else {
8         bits = 32;
9         return bits;
10    }
11 }

```

Returns the Full Operating System Name. Then, an alternate function to format the operating system name in a friendly manner.

```

1 public static string getOSFullName() {
2     return new Microsoft.VisualBasic.Devices.ComputerInfo().OSFullName.ToString();
3 }
4
5 //Returns a 'friendly' string with the OS listed.
6 public static string getFriendlyOS() {
7     OSversion = getOSFullName() + " " + getBits() + "-bit";
8     return OSversion;
9 }

```

Returns the IPv4 address of a machine.

```

1 //Returns the IPv4 that the user is using.
2 public static string getIPv4address(){
3     if (IPv4address != null){

```

```

4         return IPv4address;
5     }
6     IPAddress[] ipv4Addresses = Array.FindAll(Dns.GetHostEntry(string.Empty).AddressList,
7         ↪ a => a.AddressFamily == AddressFamily.InterNetwork);
8     int i = 1;
9     try{
10         IPv4address = ipv4Addresses[i].ToString();
11     }
12     catch (System.IndexOutOfRangeException){
13         i = 0;
14         IPv4address = ipv4Addresses[i].ToString();
15     }
16     return IPv4address;
17 }

```

Determines CPU specs for a machine.

```

1 //Sets the CPU specs for the machine.
2 ManagementObject Mo = new ManagementObject("Win32_Processor.DeviceID='CPU0'");
3 uint speed = (uint)(Mo["CurrentClockSpeed"]);
4 string name = Mo["Name"].ToString();
5 Mo.Dispose();
6 int CPUSpeed = Convert.ToInt32(speed);
7 string CPUmodel = name;

```

Determines the model of a PC.

```

1 string PCModel
2
3 System.Management.SelectQuery query = new System.Management.SelectQuery(@"Select * from
4     ↪ Win32_ComputerSystem");
5 using (System.Management.ManagementObjectSearcher searcher = new System.Management.
6     ↪ ManagementObjectSearcher(query))
7
8 foreach (System.Management.ManagementObject Mo in searcher.Get()){
9     Mo.Get();
10    string Model = Mo["Model"].ToString();
11    //System.Console.WriteLine("{0}{1}", "...System Model: ", Mo["Model"]);
12    PCModel = Model;
13 }
14 if (PCModel == "System Product Name"){
15     PCModel = "unknown";
16 }

```

Returns the installed RAM in a machine. Multiple methods are listed which give varied results depending on the environment.

```

1 //This returns an estimate of the ram installed on a machine.
2 //It is keyed to take the total bytes of RAM and convert them to the nearest 2^n value.
3 //This is the old function to determine RAM capacity. getInstalledRAM() returns a more
4     ↪ accurate value.
5 public static ulong getTotalPhysicalMemoryInBytes(){
6     return new Microsoft.VisualBasic.Devices.ComputerInfo().TotalPhysicalMemory;
7 }
8 public static ulong getTotalVirtualMemoryInBytes(){
9     return new Microsoft.VisualBasic.Devices.ComputerInfo().TotalVirtualMemory;
10 }
11
12 //Returns the physically installed RAM.
13 public static ulong getInstalledRAM(){

```

```
13  string Query = "SELECT Capacity FROM Win32_PhysicalMemory";
14  ManagementObjectSearcher searcher = new ManagementObjectSearcher(Query);
15
16  UInt64 Capacity = 0;
17  foreach (ManagementObject WniPART in searcher.Get()){
18      Capacity += Convert.ToUInt64(WniPART.Properties["Capacity"].Value);
19  }
20
21  return Capacity;
22 }
```

Java

Java is a versatile, object-oriented programming language renowned for its portability, scalability, and security features. Developed by Sun Microsystems in the mid-1990s, Java has since become a cornerstone of enterprise and web application development. One of Java's key strengths is its "write once, run anywhere" philosophy, facilitated by the Java Virtual Machine (JVM), which allows Java programs to run on any platform that supports the JVM, regardless of the underlying hardware or operating system. Java offers a rich set of libraries and frameworks for building a wide range of applications, from desktop and mobile apps to enterprise systems and web services. Its strong emphasis on object-oriented principles, robust memory management, and extensive community support make Java a popular choice for both beginner and experienced developers alike.

10.1 Java Basics

A basic Java **hello world** program follows. In this program, we define a class named HelloWorld. Inside this class, we have a main method, which is the entry point of the program. Within the main method, we use `System.out.println()` to print "Hello, World!" to the console.

```
1 public class HelloWorld {  
2     public static void main(String[] args) {  
3         // Print "Hello, World!" to the console  
4         System.out.println("Hello, World!");  
5     }  
6 }
```

HTML

HTML (Hypertext Markup Language) is a fundamental language for creating and structuring web pages. It consists of tags that define the structure and content of a document, allowing developers to organize text, images, links, and other media elements. HTML documents are hierarchical, with a tree-like structure where each element is nested within others, forming a Document Object Model (DOM). Developers can manipulate the DOM using JavaScript to dynamically update content and interact with users. HTML5, the latest version of HTML, introduced new features like semantic elements (e.g., <header>, <footer>) and multimedia support (e.g., <video>, <audio>), enhancing the capabilities for building modern web applications. Understanding HTML is essential for web developers to create accessible, well-structured, and responsive websites.

11.1 HTML Basics

The below HTML is a basic example of an HTML document structure (**hello world**). It should be saved with a '.html' extension. It starts with the document type declaration (<!DOCTYPE html>), indicating that it is an HTML5 document. Inside the <html> element, the **lang** attribute specifies the language of the document (English in this case). The document contains a <head> section where **metadata** and resources for the document are defined. In this example, it includes the **character set** (<meta charset="UTF-8">), and the title of the document (<title>Hello, World!</title>). The main content of the document is contained within the <body> element. In this example, it includes a heading (<h1>) displaying the text "Hello, World!", followed by a paragraph (<p>) with the text "This is a simple HTML file."

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <title>Hello, World!</title>
6 </head>
7 <body>
8   <h1>Hello, World!</h1>
9   <p>This is a simple HTML file.</p>
10 </body>
11 </html>
```

HTML **comments** are enclosed within '<!--' and '-->' tags. Anything between these tags is treated as a comment and is ignored by the web browser when rendering the page. Comments can span multiple lines and can be placed anywhere within the HTML document. Here's an example of an HTML comment:

```
1 <!-- This is a comment. It will not be displayed in the browser. -->
```

To add a **stylesheet** to an HTML document (for css or other elements), add a link within the head element.

```
1 <head>
2   <!-- Add icon library -->
3   <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/font-awesome@4.7.0/css/font-awesome.min.css">
4
5   <!-- Bootstrap CSS -->
6   <link rel="stylesheet" href="css/vendors/bootstrap/bootstrap.min.css">
7
8   <!-- main css -->
9   <link rel="stylesheet" href="css/styles.css">
10 </head>
```

The **div** element is a generic container that allows you to group and style content together. It's commonly used for layout purposes and to apply styling or scripting to multiple elements at once.


```
1 <div id="container">
2   <h2>This is a div container</h2>
3   <p>This is some content inside the container.</p>
4 </div>
```

The **span** element is an inline container used to mark up a part of text or a part of a document. It's often used to apply styles to specific parts of text.

```
1 <p>This is a <span style="color: red;">highlighted</span> word.</p>
```

The **img** element is used to embed images in an HTML document.

```
1 
```

The **a** element is used to create hyperlinks to other web pages, files, locations within the same page, or email addresses.

```
1 <a href="https://www.example.com">Link to Example Website</a>
```

11.2 Useful HTML scripts

CSS

CSS (Cascading Style Sheets) is a styling language used to control the visual presentation of HTML and XML documents. It allows developers to define styles for elements such as colors, fonts, layout, and spacing, separating the content from its presentation. CSS works by selecting elements in a document and applying styles to them based on rules defined by the developer. It offers various selectors and properties to target specific elements or groups of elements, enabling fine-grained control over the appearance of a web page. CSS can be applied inline within HTML elements, embedded in the `<style>` tag in the document head, or linked externally to the HTML file. With CSS, developers can create visually appealing and responsive web designs, improving user experience and accessibility across different devices and screen sizes. Understanding CSS is essential for front-end developers to create attractive and well-designed web interfaces.

URL

For a free visual reference to css, you can use the following link:

<https://cssreference.io/>

12.1 CSS Basics

CSS supports **comments** using the `/* */` syntax. Comments are ignored by the browser.

```
1  /* This is a CSS comment */
```

CSS style rules consist of a **selector** and a **declaration** block. The selector targets one or more HTML elements, and the declaration block contains one or more property-value pairs that define the styles to apply. An example of a CSS rule follows. In this example, the selector `h1` targets all `<h1>` elements, and the declaration block sets the text color to blue, font size to 24 pixels, and font weight to bold.

```
1  h1 {  
2      color: blue;  
3      font-size: 24px;  
4      font-weight: bold;  
5  }
```

CSS uses **selectors** to target specific HTML elements for styling. There are various types of selectors, including element selectors, class selectors, ID selectors, and more. For example, to style all paragraphs with a class of "intro," you can use the following. This rule will italicize the text of all paragraphs with the class "intro."

```
1  p.intro {  
2      font-style: italic;  
3  }
```

The **color** property sets the text color. It can be specified using color names, RGB values, HEX codes, or HSL values.

```
1  /* Using RGB value */  
2  body {  
3      color: rgb(51, 51, 51);  
4  }  
5  
6  /* Using HEX value */  
7  body {  
8      color: #333;  
9  }  
10  
11 /* Using color name */  
12 body {  
13     color: darkgray;  
14 }
```

To modify the **background** look, you can use the following properties:

```
1 body {  
2   background-color: #f0f0f0; /* Light gray background color */  
3   background-image: url("background.jpg"); /* Background image */  
4   background-repeat: no-repeat; /* Do not repeat the background image */  
5   background-size: cover; /* Cover the entire background */  
6   background-position: center; /* Center the background image */  
7   background-attachment: fixed; /* Fix the background image */  
8 }
```

PHP

PHP (Hypertext Preprocessor) is a server-side scripting language commonly used for web development. It enables developers to create dynamic and interactive web applications by embedding PHP code within HTML files. PHP code is executed on the server, generating HTML content that is then sent to the client's web browser. PHP offers a wide range of features for handling form data, interacting with databases, managing sessions and cookies, and performing file operations. It also supports object-oriented programming, allowing developers to create reusable and modular code. PHP integrates seamlessly with various web servers and database management systems, making it a versatile choice for building dynamic websites and web applications. Understanding PHP is essential for back-end developers to implement server-side functionality and deliver dynamic content to users.

13.1 PHP Basics

A basic PHP **hello world** program follows. In this PHP script, we use `echo` to output "Hello, World!" to the browser. The PHP tags `<?php ... ?>` indicate the beginning and end of PHP code blocks.

```
1 <?php
2 // Print "Hello, World!" to the browser
3 echo "Hello, World!";
4 ?>
```

JavaScript

JavaScript, often abbreviated as JS, is a dynamic, lightweight programming language primarily used for client-side web development. Developed by Brendan Eich at Netscape in the mid-1990s, JavaScript has evolved into a versatile and essential tool for creating interactive and dynamic web content. Unlike Java, which is primarily used for server-side programming, JavaScript is executed on the client's browser, enabling developers to enhance user experiences with features such as dynamic content updates, form validation, and interactive elements. JavaScript is supported by all modern web browsers and is often used in conjunction with HTML and CSS to create modern, responsive web applications. Its flexibility, simplicity, and widespread adoption have made JavaScript an integral part of web development, powering everything from simple webpage animations to complex single-page applications.

Python

Python is a high-level, interpreted programming language known for its simplicity, readability, and versatility. With its elegant syntax and dynamic typing, Python facilitates rapid development and prototyping across various domains, including web development, data analysis, artificial intelligence, and scientific computing. Its extensive standard library and vibrant ecosystem of third-party packages provide comprehensive support for a wide range of tasks, from basic scripting to complex software development. Python's object-oriented and functional programming paradigms, coupled with its strong community support and cross-platform compatibility, make it an ideal choice for both beginners and experienced developers seeking efficient and expressive solutions to their programming challenges. Its emphasis on code readability and simplicity distinguishes Python from other languages, promoting maintainability and collaboration in software projects.

The official python documentation can be found at the following links

```
1 # Documentation for version 3+
2 https://docs.python.org/3/
3 # Documentation for version 2+
4 https://docs.python.org/2/
```

15.1 Basics of Python

15.1.1 Imports and libraries

Import floating point division which allows python 2 compatibility when using division with doubles. Include this at the beginning of the script.

```
1 from __future__ import division
```

On Linux, there's a site-packages folder that imported libraries are all in. These are located in the following directories for python versions 2.7 and 3.6 respectively.

- /usr/lib/python2.7/site-packages
- /usr/lib/python3.6/site-packages

I believe you only have specify the path from that folder on. For example, an app that uses the import program.script.interface as interface will have the interface.py file installed to usr/lib/python3.6/site-packages/program/scripts.

15.1.2 Program parameters

You can add input parameters for a python script/program using the argparse package.

```
1 import argparse
2
3 parser.add_argument(
4     '-v',                                # Creates a parameter with -v
5     '--verbose',                          # Creates args.verbose parameter
6     required=False,                       # Sets the parameter to not required.
7     default=False,                       # Sets default value
8     action='store_true',                 # Sets value to true if supplied
9     help="Enables verbose mode.")      # Sets help message
10
11 parser.add_argument(
12     '-i',                                # Creates a parameter with -i
13     '--input',                            # Creates args.input parameter
14     required=True,                       # Sets the parameter to be required
15     default="potato",                    # Sets default value to "potato"
```

```
16         help="An input string to use") # Sets a help message
17
18 if args.input is not None:
19     print("Input value is {}".format(args.input)) # prints the input value.
20
21 if args.verbose:
22     print("Verbose is on!")
```

15.1.3 Methods, Functions, and Classes

To define a **function** in Python, you use the **def** keyword followed by the function name and parameters, if any. You can also include a docstring to provide documentation for the function. In this example, the `greet()` function takes a `name` parameter and returns a greeting message.

```
1 def greet(name):
2     """
3     This function greets the user.
4     :param name: The name of the person to greet
5     :return: A greeting message
6     """
7     return f"Hello, {name}!"
```

Methods are functions that are associated with objects. They are defined within the scope of a class and are accessed through instances of the class. In this example, the `bark()` method of the `Dog` class prints a bark message when called.

```
1 class Dog:
2     def bark(self):
3         """
4         This method makes the dog bark.
5         """
6         print("Woof!")
```

Classes in Python allow you to define your own data types with custom attributes and methods. In this example, the `Rectangle` class represents a geometric rectangle. It has an `__init__()` method to initialize its width and height attributes and an `area()` method to calculate its area.

```
1 class Rectangle:
2     def __init__(self, width, height):
3         self.width = width
4         self.height = height
5
6     def area(self):
7         return self.width * self.height
```

To use the `Rectangle` class, you can create an instance of it by passing the width and height values as parameters to the constructor:

```
1 # Constructing a Rectangle object
2 rectangle1 = Rectangle(5, 10)
3
4 # Calculating the area of the rectangle
5 area = rectangle1.area()
6
7 print("The area of the rectangle is:", area)
```

15.1.4 String manipulation

To split a string by a **delimiter** you can use the **split()** method. To remove any leading or trailing **whitespace** from a string, you can use the **strip()** method. Additionally, you can use **lstrip()** to remove leading whitespace and **rstrip()** to remove trailing whitespace.

```

1 stringValue = "Test; string "
2
3 firstPart = stringValue.split(';')[0]    # Stores "Test"
4 secondPart = stringValue.split(';')[1]    # Stores " string "
5 noWhiteSpace = secondPart.strip()         # Stores "string"
6 leftWhiteSpace = secondPart.lstrip()      # Stores "string "
7 rightWhiteSpace = secondPart.rstrip()     # Stores " string"

```

The opposite of splitting strings is joining them. You can use the **join()** method to concatenate a sequence of strings into a single string, using a specified delimiter.

```

1 words = ['Hello', 'world', '!']
2
3 # Join the words with a space delimiter
4 sentence = ' '.join(words)    # Stores "Hello world !"

```

Python provides methods to convert the case of strings. You can use **lower()** to convert a string to lowercase and **upper()** to convert it to uppercase.

```

1 text = "Hello, World!"
2
3 # Convert the string to lowercase
4 lowerCaseText = text.lower()    # Stores "hello, world!"
5
6 # Convert the string to uppercase
7 upperCaseText = text.upper()    # Stores "HELLO, WORLD!"

```

To insert variables into a string, you can use the **format()** method or by using an **f-string** (introduced in python3.6).

```

1 a = 5
2 b = 6
3 sum = a + b
4 equation = "{0} + {1} = {2}".format(a, b, sum)
5 f_equation = f"{a} + {b} = {sum}"
6
7 print(equation)    # prints "5 + 6 = 11"
8 print(f_equation) # prints "5 + 6 = 11"

```

15.1.5 Arrays and lists

To check if any value in one array or list is present in another array or list, you can use the **any()** function along with a generator expression. In this example, the **any()** function iterates over each item in **list2** and checks if it is present in **list1**. If any common elements are found, the condition evaluates to **True**, and the corresponding message is printed.

```

1 list1 = ["apple", "banana", "orange"]
2 list2 = ["orange", "grape", "pear"]
3
4 if any(item in list1 for item in list2):
5     print("Common elements found between list1 and list2")
6 else:
7     print("No common elements found")

```


You can concatenate two or more lists using the **+** operator or the **extend()** method. Both methods produce the same result: a new list containing all the elements from the original lists in the specified order.

```

1 list1 = [1, 2, 3]
2 list2 = [4, 5, 6]
3
4 # Using the + operator
5 concatenated_list = list1 + list2    # Stores [1, 2, 3, 4, 5, 6]
6
7 # Using the extend() method
8 list1.extend(list2)    # Modifies list1 to [1, 2, 3, 4, 5, 6]

```

To find the number of elements in a list, you can use the built-in **len()** function.

```

1 my_list = [10, 20, 30, 40, 50]
2
3 # Find the length of the list
4 list_length = len(my_list)    # Stores 5

```

15.1.6 Plotting and Graphs

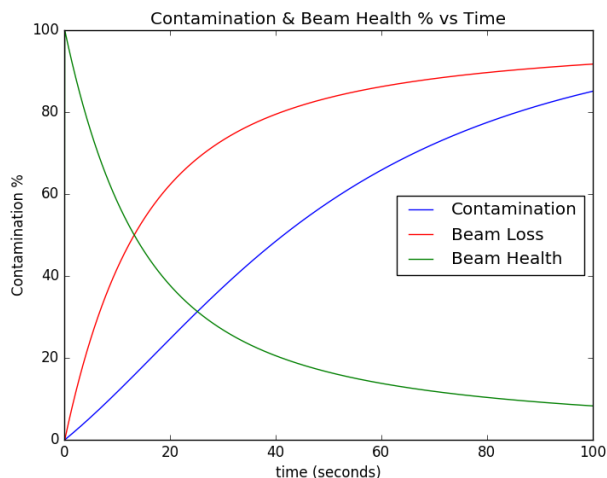
A nicely formatted plot with a legend using the pylab package.

```

1 import pylab as plt #Imports the correct packages for plotting.
2
3 plt.title('Contamination & Beam Health % vs Time')    # Creates a title.
4
5 plt.plot(t, Contamination, '-b', label='Contamination') # Plots Contamination in blue.
6 plt.plot(t, Beam_loss, '-r', label='Beam Loss')    # Plots Beam_loss in red.
7 plt.plot(t, Beam_health, '-g', label='Beam Health') # Plots Beam_health in green.
8
9 plt.xlabel("time (seconds)")    # Creates a x-axis label
10 plt.ylabel("Contamination %")    # Creates a y-axis label
11
12 plt.legend(loc='center right') # Creates a legend with the labels set above.
13 # Other locations include upper/lower/center left/right
14
15 plt.show() # Displays plot.

```

This code would display a graph such as the one below such that the proper values are input.



A nicely formatted plot with a legend using the matplotlib package.

```

1 import matplotlib as plt      # Imports the correct packages for plotting.
2
3 fig = plt.figure(dpi=1200)    # Increase resolution of the plot.
4 plt.scatter(x, y, s=0.1)      # Plot x data vs y data with a dot size of 0.1.
5 fig.suptitle(title, fontsize=12) # adds a title to the figure.
6
7 plt.xlabel("x axis label")    # Creates a x-axis label.
8 plt.ylabel("y axis label")    # Creates a y-axis label.
9
10 manage = plt.get_current_fig_manager()
11 manage.full_screen_toggle()   # Makes the plt full screen.
12
13 plt.show()                   # Displays plot.
14 plt.savefig("fileName.png")   # Saves the figure as an image

```

15.2 Pytest

Pytest is a powerful and popular testing framework for Python that simplifies the process of writing and executing tests. It offers a straightforward syntax and extensive features for writing test cases, including assertions, fixtures, parameterization, and test discovery. Pytest provides robust support for testing different types of applications, including web applications, APIs, and command-line utilities. It promotes efficient testing practices by emphasizing readability, scalability, and flexibility, making it a preferred choice for many Python developers.

To run a python test file using pytest, you can call pytest directly via a terminal.

```

1 pytest -vv test_my_module.py

```

To write tests for a function, you must import that function and write a test for it. The **assert** is a basic test keyword is used to check if a statement is true or false.

```

1 # Code implementation in a Python file (e.g., my_module.py)
2
3 def add(x, y):
4     """Function to add two numbers."""
5     return x + y

```

```

1 # Test case using Pytest (e.g., test_my_module.py)
2
3 import pytest
4 import my_module
5
6 def test_add():
7     """Test case for the add function."""
8
9     # Test with positive integers
10    assert my_module.add(2, 3) == 5
11
12    # Test with negative integers
13    assert my_module.add(-2, -3) == -5
14
15    # Test with zero
16    assert my_module.add(0, 0) == 0
17
18    # Test with one positive and one negative integer
19    assert my_module.add(5, -3) == 2

```

A pytest **fixture** can be used to create a **temporary directory** for testing.

```

1 import pytest
2
3 @pytest.fixture
4 def temp_directory():
5     """
6     Fixture to create a temporary directory for testing.
7
8     This fixture creates a temporary directory using tempfile.mkdtemp().
9     The temporary directory path is provided to the test functions, and after testing,
10    the temporary directory is removed to clean up resources.
11
12    Yields:
13        str: Path of the temporary directory.
14    """
15    # Create a temporary directory for testing
16    temp_dir = tempfile.mkdtemp()
17    # Provide the temporary directory to the test for its duration.
18    yield temp_dir
19
20 # Call a method using the temp_directory() method.
21 def test_method(temp_directory):
22     print(temp_directory) # Prints the temp directory path

```

A pytest **fixture** can be used to return an instance of a class.

```

1 import pytest
2 from mmorpdnd import MMORPDND
3
4 @pytest.fixture
5 def mmorpdnd_instance():
6     """
7     Fixture to provide an instance of the MMORPDND class for testing.
8     """
9     return MMORPDND()

```

You can define a list of test **parameters** to be input into a test via the `@pytest.mark.parametrize` decorator, allowing multiple input-output pairs to be tested with the same test function.

```

1 @pytest.mark.parametrize("input_string, expected_output", [
2     ('6 (barbarian 3, rogue 3)', 6),
3     ('12 dwarfs and 3 elves', 12),
4     ('No numbers here!', None),
5     ('Only one number: 42', 42),
6     ('Negative number: -10', -10),
7     ('Integers: 1 2 3', 1),
8     ('999', 999),
9 ])
10 def test_extract_first_integer(input_string, expected_output):
11     """
12     Test case to verify the behavior of extract_first_integer function.
13     """
14     assert extract_first_integer(input_string) == expected_output

```

15.3 Python tk GUI

Python **Tkinter** (**Tk**) is a built-in **GUI** (Graphical User Interface) toolkit that allows developers to create desktop applications with a graphical interface in Python. Tkinter provides a set of widgets (such as buttons, labels, and entry

boxes) and tools to arrange them within windows and frames. It is based on the Tk GUI toolkit originally developed for the Tcl programming language. With Tkinter, developers can create cross-platform applications that run on Windows, macOS, and Linux, making it a popular choice for simple to moderately complex desktop applications in Python. To install Tk on linux (APT), the following command can be used.

```
1 sudo apt-get install python3-tk
```

A basic **hello world** gui written in Tk follows:

```
1 import tkinter as tk
2
3 def say_hello():
4     label.config(text="Hello, World!")
5
6 # Create the main application window
7 gui = tk.Tk()
8 gui.title("Hello, Tkinter!")
9
10 # Create a label widget
11 label = tk.Label(gui, text="Click the button to say hello!")
12 label.pack(pady=10)
13
14 # Create a button widget
15 button = tk.Button(gui, text="Say Hello", command=say_hello)
16 button.pack()
17
18 # Run the Tkinter event loop
19 gui.mainloop()
```

To change the **geometry** of a Tk gui window, the following can be used:

```
1 gui = tk.Tk()
2 gui.geometry("300x470")
```

To change the **icon** of a Tk gui window, the following can be used:

```
1 # Load icon image
2 icon = PhotoImage(file=f'{root_dir}/img.png')
3 # Set icon image
4 gui.tk.call('wm', 'iconphoto', self.gui._w, icon)
```

15.4 Useful Methods and Packages

15.4.1 Useful Packages

In this section, we explore Python's extensive collection of packages, offering tailored solutions for diverse tasks. Python's packages are specialized tools, empowering developers to address specific challenges efficiently. From data manipulation to machine learning, web development to scientific computing, Python packages provide a robust foundation for diverse projects.

15.4.1.1 collections

Python's **collections** module provides a **Counter** class, which is a specialized dictionary designed for counting hashable objects. One lesser-known feature of Counter is that it supports arithmetic operations like **addition**, **subtraction**, **intersection**, and **union**. This is particularly useful when you're dealing with counting occurrences of items across different datasets or need to perform set-like operations on the counts themselves.

```

1 from collections import Counter
2
3 # Define two Counters
4 counter1 = Counter({'a': 3, 'b': 1, 'c': 2})
5 counter2 = Counter({'a': 1, 'b': 2, 'd': 1})
6
7 # Addition: Adds counts from two counters
8 print("Addition:", counter1 + counter2)
9 # Prints "Addition: Counter({'a': 4, 'b': 3, 'c': 2, 'd': 1})"
10
11 # Subtraction: Subtracts counts, but keeps only positive results
12 print("Subtraction:", counter1 - counter2)
13 # Prints "Subtraction: Counter({'a': 2, 'c': 2})"
14
15 # Intersection: Keeps only positive counts common to both counters
16 print("Intersection:", counter1 & counter2)
17 # Prints "Intersection: Counter({'a': 1, 'b': 1})"
18
19 # Union: Keeps maximum counts from both counters
20 print("Union:", counter1 | counter2)
21 # Prints "Union: Counter({'a': 3, 'b': 2, 'c': 2, 'd': 1})"

```

Python's **collections** module provides a class called **defaultdict**, which is a subclass of the built-in dictionary (**dict**) class. The **defaultdict** is similar to a regular dictionary, but it allows you to specify a default value factory for missing keys. This means that when you access a key that doesn't exist, instead of raising a **KeyError**, **defaultdict** will create the key and assign it a default value returned by the factory function.

```

1 from collections import defaultdict
2
3 # Create a defaultdict with int as the default value factory
4 d = defaultdict(int)
5
6 # Accessing a non-existent key will create it with a default value of 0
7 print(d['a']) # Output: 0
8
9 # You can also specify a different default value factory
10 d = defaultdict(list)
11
12 # Accessing a non-existent key will create it with an empty list as the default value
13 print(d['b']) # Output: []
14
15 # You can use any callable as the default value factory
16 d = defaultdict(lambda: 'default')
17
18 # Accessing a non-existent key will create it with 'default' as the default value
19 print(d['c']) # Output: 'default'

```

15.4.1.2 itertools

Python's **itertools** module provides functions called **combinations** and **combinations_with_replacement**, which generates all possible **combinations** of a given length from the elements of an **iterable**, including combinations with repeated elements.

```

1 from itertools import combinations
2
3 # Generate combinations of length 2 from the elements 'A', 'B', 'C' without replacement
4 combinations = combinations(['A', 'B', 'C'], 2)
5

```

```

6 # Print the generated combinations
7 for combo in combinations:
8     print(combo, end=',')
9 # Prints "('A', 'B'),('A', 'C'),('B', 'C'),"

1 from itertools import combinations_with_replacement
2
3 # Generate combinations of length 2 from the elements 'A', 'B', 'C' with replacement
4 combinations = combinations_with_replacement(['A', 'B', 'C'], 2)
5
6 # Print the generated combinations
7 for combo in combinations:
8     print(combo, end=',')
9 # Prints "('A', 'A'),('A', 'B'),('A', 'C'),('B', 'B'),('B', 'C'),('C', 'C'),"

```

15.4.1.3 slice

Python has a built-in **slice** object that can be used to slice **sequences** like **lists**, **tuples**, and **strings**. While slicing with regular slicing syntax (`list[start:end:step]`) is common, creating and using slice objects directly can be powerful, especially when working with multiple slices or when you want to reuse the same slice configuration.

```

1 # Create a slice object with start, stop, and step
2 my_slice = slice(1, 5, 2)
3
4 # Apply the slice to a list
5 my_list = ['a', 'b', 'c', 'd', 'e', 'f', 'g']
6 sliced_result = my_list[my_slice]
7
8 print(sliced_result) # Output: ['b', 'd']
9
10 # You can also use the slice object with other sequences like strings
11 my_string = "Hello, World!"
12 sliced_string = my_string[my_slice]
13
14 print(sliced_string) # Output: 'el'

```

In this example, `slice(1, 8, 2)` creates a **slice** object that starts at index 1 ('b'), ends at index 8 ('i'), and steps by 2. So, applying this slice to the list `['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i']`, it selects elements at indices 1, 3, 5, and 7, which correspond to the values 'b', 'd', 'f', and 'h', respectively.

```

1 # Create a slice object with start, stop, and step
2 my_slice = slice(1, 8, 2)
3
4 # Apply the slice to a list
5 my_list = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i']
6 sliced_result = my_list[my_slice]
7
8 print(sliced_result) # Output: ['b', 'd', 'f', 'h']

```

15.4.1.4 functools

Python's **functools** module provides a function called **singledispatch**, which allows you to create a single-dispatch generic function. It allows you to define a function behavior based on the type of the first argument. This is particularly useful for creating functions that can handle different types of input in a flexible and extensible way, similar to **method overloading** in other programming languages.

```

1 from functools import singledispatch
2
3 @singledispatch
4 def my_func(arg):
5     print("Default implementation:", arg)
6
7 @my_func.register(int)
8 def _(arg):
9     print("Processing an integer:", arg)
10
11 @my_func.register(str)
12 def _(arg):
13     print("Processing a string:", arg)
14
15 @my_func.register(list)
16 def _(arg):
17     print("Processing a list:", arg)
18
19 # Test the function with different types of input
20 my_func(10)           # Output: Processing an integer: 10
21 my_func("Hello")      # Output: Processing a string: Hello
22 my_func([1, 2, 3])    # Output: Processing a list: [1, 2, 3]

```

Python's **functools** module provides a decorator called **lru_cache**, which stands for "Least Recently Used Cache." It's a built-in memoization decorator that caches the results of a function and reuses them when the same inputs occur again. **lru_cache** Uses a least-recently-used (LRU) eviction policy. This means that when the cache reaches its maximum size (specified by the `maxsize` argument), the least recently used entries are discarded to make room for new ones.

```

1 @lru_cache(maxsize=None)
2 def fib(n):
3     if n < 2:
4         return n
5     return fib(n-1) + fib(n-2)
6
7 >>> [fib(n) for n in range(16)]
8 [0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610]
9
10 >>> fib.cache_info()
11 CacheInfo(hits=28, misses=16, maxsize=None, currsize=16)

```

Python's **functools** module provides a function called **cache**, which allows you to cache the results of a function call and reuse them when the same inputs occur again. This can significantly improve the performance of your code, especially when dealing with expensive computations or I/O operations. This was introduced in python 3.9. Unlike **lru_cache**, **cache** Does not have a built-in eviction policy. It simply caches results until the program exits or until the function is explicitly uncached. Because it never needs to evict old values, this is smaller and faster than **lru_cache** with a size limit.

```

1 from functools import cache
2
3 @cache
4 def factorial(n):
5     return n * factorial(n-1) if n else 1
6
7 >>> factorial(10)           # no previously cached result, makes 11 recursive calls
8 3628800
9 >>> factorial(5)           # just looks up cached value result
10 120

```

```

11 >>> factorial(12)          # makes two new recursive calls, the other 10 are cached
12 479001600

```

15.4.1.5 typing

Python's **typing** module provides a way to specify type hints for variables, function arguments, and return values. While type hints are often used for improving code readability and catching errors early during development, you can also use them for **runtime** type checking using the typing module's **runtime_checkable** decorator along with the **Protocol** class. In this example, Printable is defined as a protocol using the Protocol class. It specifies a method print that takes no arguments and returns None. By decorating Printable with @runtime_checkable, you enable runtime type checking for objects that claim to conform to the protocol. The print_if_possible function takes an argument of type Printable and calls its print method. When called with an object that conforms to the Printable protocol (like Printer), it works fine. However, if called with an object that does not conform to the Printable protocol (like NotPrinter), it raises a TypeError, allowing you to catch potential type errors at runtime.

```

1  from typing import Protocol, runtime_checkable
2
3  # Define a protocol with type hints for methods
4  @runtime_checkable
5  class Printable(Protocol):
6      def print(self) -> None:
7          pass
8
9  # A class that conforms to the Printable protocol
10 class Printer:
11     def print(self) -> None:
12         print("Printing...")
13
14 # A class that does not conform to the Printable protocol
15 class NotPrinter:
16     def display(self) -> None:
17         print("Displaying...")
18
19 def print_if_possible(obj: Printable) -> None:
20     obj.print()
21
22 printer = Printer()
23 not_printer = NotPrinter()
24
25 # This will work fine since Printer conforms to the Printable protocol
26 print_if_possible(printer)
27
28 # This will raise a TypeError since NotPrinter does not conform to the Printable protocol
29 print_if_possible(not_printer)

```

15.4.1.6 enum

Python's **enum** module provides a powerful way to create **enumerations** in Python. Enumerations allow you to define symbolic names (members) for a set of unique values, which can make your code more readable and maintainable. One lesser-known feature of enumerations is the ability to create them dynamically using the Enum class's functional API. This allows you to create enumerations without explicitly subclassing Enum.

```

1  from enum import Enum
2
3  # Define a dynamic enumeration
4  DynamicEnum = Enum("DynamicEnum", ["VALUE1", "VALUE2", "VALUE3"])
5

```



```

6 # Access enumeration members
7 print(DynamicEnum.VALUE1) # Output: DynamicEnum.VALUE1
8 print(DynamicEnum.VALUE2) # Output: DynamicEnum.VALUE2
9 print(DynamicEnum.VALUE3) # Output: DynamicEnum.VALUE3

```

To access an **enum** name from an **enum** class based on an the value, you can write a method to access the enum values and compare to another value.

```

1 from enum import Enum
2
3 class Color(Enum):
4     RED = 1
5     GREEN = 2
6     BLUE = 3
7
8 def get_enum_from_value(value):
9     for color in Color:
10         if color.value == value:
11             return color
12     raise ValueError("No enum member with that value")
13
14 color = get_enum_from_value(2)
15 print(color) # Output: Color.GREEN

```

15.4.1.7 contextlib

Python's **contextlib** module provides a decorator called **contextmanager**, which allows you to create context managers using generator functions. Context managers are objects that define `__enter__` and `__exit__` methods, and they are commonly used with the `with` statement to manage resources, such as opening and closing files or acquiring and releasing locks.

```

1 from contextlib import contextmanager
2
3 @contextmanager
4 def open_file(filename, mode):
5     file = open(filename, mode)
6     try:
7         yield file
8     finally:
9         file.close()
10
11 # Usage of the context manager
12 with open_file("example.txt", "w") as f:
13     f.write("Hello, context managers!")

```

Python's **contextlib** module provides utilities for working with context managers. One lesser-known feature of the **contextlib** module is the **redirect_stdout** and **redirect_stderr** context managers, which allow you to temporarily redirect the standard output and standard error streams to a specified file or file-like object.

```

1 from contextlib import redirect_stdout
2 import io
3
4 # Create a StringIO object to redirect output
5 output_buffer = io.StringIO()
6
7 # Redirect stdout to the StringIO object
8 with redirect_stdout(output_buffer):
9     print("This will be captured by the redirect_stdout context manager")
10

```

```

11 # Get the captured output from the StringIO object
12 captured_output = output_buffer.getvalue()
13
14 print("Captured Output:", captured_output)
15 # Prints "Captured Output: This will be captured by the redirect_stdout context manager"

```

15.4.2 Useful Methods

This section is a collection of useful methods that can be adapted or used for many applications. These are all methods that I've written or heavily modified for my own use and have found useful on multiple occasions.

15.4.2.1 Logging and Application Output

To perform **logging** for an application, you often want to create a **log file** named based on the application name and the current date. This can be expanded to be contained in a class and called during initialization or just used as standalone.

```

1  #!/bin/python3
2  import os
3  import sys
4
5  # Used for gathering time and date data.
6  from datetime import datetime
7
8
9  def set_log_file_name():
10     """
11     Returns the log file name. If it has not yet been set, creates the log filename that
12     ↪ contains the date and then returns it.
13
14     The desired behavior is that one log_file is used per run of the application.
15     To account for day changes while the program is running, log_file must be created at
16     ↪ application runtime.
17     """
18     global log_file
19
20     # Get the script name with full path
21     script_path = os.path.abspath(sys.argv[0])
22     # Get the script name without the extension.
23     script_name = os.path.basename(script_path).split('.py')[0]
24
25     # Check if the log file already has been set.
26     if log_file == "":
27         today = datetime.today().strftime("%Y%m%d")
28         log_file = f"{script_name}_log_{today}"
29
30     return log_file
31
32 # The log file to be used throughout the file.
33 log_file = ""
34 set_log_file_name()

```

With a **log file** defined, you can perform **logging** by appending messages to a log file. The message can include a **time stamp** for referencing when various tasks were performed at a later date.

```

1  # Used for gathering time and date data.
2  from datetime import datetime
3

```

```

4
5 def log_text(text, include_time_stamp=True):
6     """
7     This will output text to the logfile with an optional time stamp.
8     Args:
9         text (str): The text to be logged.
10        timestamp (bool): Option to include timestamp in logging.
11    """
12    # Set's the appropriate timestamp value to use.
13    if include_time_stamp:
14        timestamp_text = datetime.now().strftime("[%Y-%m-%d %H %M %S]")
15    else:
16        timestamp_text = ""
17
18    # Logs the output. This assumes log_file is defined.
19    with open(log_file, 'a') as log:
20        log.write(f"{timestamp_text} {text}\n")

```

When writing an application with complex or varied output, it is often useful to **print** messages to the terminal using various **color codes**. To do this, you can write a method that takes either various types of options (each corresponding to a color code) or takes a color name directly. You can also modify it to optionally automatically log the information using the `log_text()` method defined above. This is a highly adaptable code snippet.

```

1 def output_text(text:str, option:str=None , color:str=None , log=True):
2     """
3     Print text in different colors based on the provided option. This will also log the
4     ↪ output text (including a time stamp).
5
6     Args:
7         text (str): The text to be printed and logged.
8         option (str): The color option for the text. Valid options are defined in the
9         ↪ color mappings list (second element). Default = None.
10        color (str): The color name for the text. Valid options are defined in the color
11        ↪ mappings list (first element). Default = None.
12        log (bool): Determines whether to log the text or not. Default = True.
13
14    Returns:
15        None
16
17    Note:
18        This function uses ANSI escape codes for color formatting. Colors may not be
19        ↪ displayed correctly in all environments. The color and option parameters
20        ↪ should not both be used. If they are, the color will override the option
21        ↪ parameter.
22    """
23    # Defines the default behaviour.
24    if color == None and option == None:
25        color = "white"
26    if color != None and option != None:
27        option = None
28    if color == None:
29        color = ""
30    if option == None:
31        option = ""
32
33    # Define the list of color mappings
34    color_mappings = [
35        ("white", "text", "\033[0m"), # White/default/reset
36        ("red", "error", "\033[91m"), # Red

```

```

31         ("green",    "success", "\033[92m)",    # Green
32         ("yellow",   "warning", "\033[93m)",    # Yellow
33         ("blue",     "note",    "\033[94m)",    # Blue
34     ]
35
36     # Set's the reset value for normal text.
37     reset_code = "\033[0m"
38     found_match = False
39
40     for color_name, color_type, color_code in color_mappings:
41         if color.lower() == color_name:
42             print(f"{color_code}{text}{reset_code}")
43             found_match = True
44             break
45         elif option.lower() == color_type:
46             print(f"{color_code}{text}{reset_code}")
47             found_match = True
48             break
49
50     # If no color match was found, print the text.
51     if not found_match:
52         print(text)
53
54     if log:
55         log_text(text)

```

15.1: ANSI Color Codes For Terminal Output

The American National Standards Institute (**ANSI**) **color codes** in the above can be expanded further if the application needs more color outputs. The below chart can be used.

Color Name	ANSI Escape Code	Example
White	\033[0m	
Black	\033[30m	Text
Red	\033[91m	Text
Green	\033[92m	Text
Yellow	\033[93m	Text
Blue	\033[94m	Text
Magenta	\033[35m	Text
Cyan	\033[36m	Text
Gray	\033[90m	Text
Purple	\033[95m	Text
Orange	\033[33m	Text

15.4.2.2 Config Files

For a basic and somewhat generic **config file framework**, you can use the following setup. The config variables and default values are stored in a map. This method will then update any variable/value pairs that are found in a configuration file. The config file should be setup so that each line contains a 'variable=value' pair. The value's must match the type that are defined in the configuration variables map. Lines that start with a '#' are considered comment lines and ignored. The values can then all be called by calling the configuration map.

```

1 # Example config file setup for the below framework.
2 name = John
3 age = 30

```

```

4 weight = 70.5

1 # Example config_variables
2 config_vars = {
3     'name':          'default_name',
4     'age':           0,
5     'weight':        0.0,
6     'is_student':    False,
7     'favorite_color': 'red'
8 }
9
10 def load_config_file(config_file, config_variables, verbose=False):
11     """
12     Load configuration variables from a file and update their values.
13
14     Args:
15         config_file (str): The path to the configuration file.
16         config_variables (map): The map of configuration variables.
17             Should be formatted with default values for each possible configuration
18             ↪ variable desired:
19             {'name': 'default', 'age': 0, 'weight': 0.0, ...}
20         verbose (bool, optional): If True, print verbose messages indicating the
21             ↪ configuration variables being set.
22             Defaults to False.
23
24     Returns:
25         None
26
27     Raises:
28         FileNotFoundError: If the specified config_file is not found.
29
30     Notes:
31         - Configuration variables are updated based on the values specified in the config
32           ↪ file.
33         - The config file should have lines in the format "variable = value".
34         - Each variable's value will be converted to the specified type as per the
35           ↪ config_variables dictionary.
36     """
37     try:
38         with open(config_file, 'r') as file:
39             for line in file:
40                 # Skip commented lines.
41                 if line.startswith('#'):
42                     continue
43
44                 if '=' in line:
45                     variable, value = map(str.strip, line.strip().split('='))
46                     if variable in config_variables:
47                         # Retrieve default value and type
48                         value_type = type(config_variables[variable])
49                         # Convert the value to the proper type
50                         converted_value = value_type(value)
51                         config_variables[variable] = converted_value
52                         if verbose:
53                             print(f"Setting config variable: {variable} -> {
54                                 ↪ converted_value}")
55                     elif verbose:
56                         print(f"WARNING: Invalid variable detected in config file: {
57                             ↪ variable}")

```

```
52     except FileNotFoundError:  
53         print("ERROR: Config file not found.")
```

References

- [1] <http://www.cplusplus.com/doc/tutorial/pointers/>
- [2] <http://www.cplusplus.com/reference/chrono/>
- [3] <https://os.mbed.com/handbook/C-Data-Types>
- [4] Kumar, Chandan. “10 Useful Linux Networking Commands.” Geek Flare, 11 Feb. 2018, geekflare.com/linux-networking-commands/.
- [5] Parker, Steve. “Shell Scripting Tutorial.” The Shell Scripting Tutorial, www.shellscript.sh/.
- [6] <https://git-scm.com/docs/>
- [7] <https://www.toptal.com/git/the-advanced-git-guide>

Index

- + operator, 47
- .gitignore, 27
- .gitmodules, 29
- __init__, 45

- a, 39
- add, 26
- addition, 50
- address, 13
- address-of, 13
- amend, 28
- ampersand, 13
- ANSI, 58
- any, 46
- apt, 2
- arguments, 12
- arithmetic, 8
- assert, 48
- author, 31
- authors, 30

- background, 41
- backupp, 4
- Bash, 6
- big endian, 20
- bisect, 28
- boolean, 14
- brace expansion, 8
- branch, 27

- C++, 12
- C#, 33
- c_str, 16
- cache, 53
- cd, 2
- Changing directory, 2
- character set, 38
- checkout, 27
- chgrp, 5
- chmod, 5, 7
- chown, 5, 7
- Class, 15
- Classes, 45
- classes, 12
- clone, 26
- CMake, 24
- CMakeLists, 24
- collections, 50, 51
- color, 40
- color codes, 57, 58
- combinations, 51
- combinations_with_replacement, 51
- comments, 30, 38, 40
- commit, 26, 28
- commit message, 28
- compile-time, 20
- config file, 58
- Config Files, 58
- configuration, 26
- const char*, 16, 18
- constexpr, 20
- contextlib, 55
- contextmanager, 55
- Counter, 50
- CSS, 40
- current directory, 2

- declaration, 40

- def, 45
- defaultdict, 51
- delimiter, 46
- diff, 26
- div, 38
- dnf, 2
- Domain Information Groper, 5
- domain-specific, 21
- double, 16

- echo, 7
- Emacs, 9
- enum, 54, 55
- enumerations, 54
- export, 7
- extend, 47

- f-string, 46
- fetch, 26
- Fibonacci, 17
- find, 3
- fixture, 48, 49
- float, 16
- for loop, 8
- force-with-lease, 28
- foreach, 29
- format, 46
- framework, 58
- function, 45
- Function Pointers, 22
- function pointers, 22
- functools, 52, 53

- geometry, 50
- git, 26
- Git Submodules, 28
- GitHub, 29
- GitHub Pages, 29
- God Mode, 10
- GUI, 49

- HEAD, 29
- head, 4
- hello world, 12, 37, 38, 42, 50
- HTML, 38
- htop, 2
- hyperlinking, 32

- icon, 50
- ifconfig, 5
- ifdown, 5
- ifup, 5
- img, 39
- inheritance, 12
- init, 26
- Input and Output, 14
- integer, 16
- intersection, 50
- iotop, 2
- IPv6, 6
- iterable, 51
- itertools, 51

- Java, 37
- join, 46

- Key Strokes, 18, 19
- KeyError, 51

- lang, 38
- LaTeX, 30
- len, 47
- Linux, 19, 20
- lists, 52
- literals, 21
- little endian, 20
- log, 26
- log file, 56
- Logging, 56
- logging, 56
- long double, 16
- long long int, 16
- lower, 46
- lru_cache, 53
- lstrip, 46

- Make, 24
- memory management, 12
- merge, 26, 27
- mergetool, 27
- metadata, 38
- method overloading, 52
- Methods, 45
- MiKTeX, 30
- mkfifo, 8
- Mount, 4
- mount, 4
- mouse, 10

- netstat, 5
- Networking, 5
- nmap, 5
- nslookup, 5

- object-oriented, 12
- OOP, 12
- Output, 56

- package, 30
- parameters, 49
- pdf, 4
- PHP, 42
- Plotting and Graphs, 47
- Pointers, 13
- pointers, 22
- polymorphism, 12
- prime, 17
- print, 57
- Protocol, 54
- pull, 26
- push, 26
- Pytest, 48
- pytest.mark.parametrize, 49
- Python, 44

- read-only mode, 9
- rebase, 28
- redirect.stderr, 55
- redirect.stdout, 55
- remote, 28
- Remote Connections, 3
- remote flag, 29
- reset, 28
- rstrip, 46
- rsync, 3, 4
- runtime, 20, 54
- runtime.checkable, 54

scp, 3
screen, 3
Secure Shell, 3
selector, 40
selectors, 40
sequences, 52
services, 10
sftp, 3
shebang, 6
singledispatch, 52
sleep, 18
slice, 52
span, 39
split, 46
SSH, 3
stash, 27
status, 26
std::string, 16
std::to_string, 16
stderr, 6
stdout, 6
stod, 16
stof, 16
stoi, 16
stold, 16
stoll, 16

stoul, 16
strings, 52
strip, 46
stylesheet, 38
su, 5
submodules, 28
subtitle, 30
subtitles, 30
subtraction, 50
System File Checker, 10
System Related Commands, 2

tail, 4
tar, 4
telnet, 5
template, 22
template specialization, 22
Templates, 14
templates, 22
temporary directory, 48
TeX Live, 30
time stamp, 56
titles, 30
Tk, 49
Tkinter, 49
tmux, 3
top, 2

touch, 8
traceroute, 5
tuples, 52
typing, 54

union, 50
unmount, 4
unsigned int, 16
upper, 46
user-defined, 21
useradd, 4

Variable Types, 15
variables, 7
Vector, 15
vector, 15
VNC, 3

wc, 8
whitespace, 46
Windows, 18, 20

X11 forwarding, 3
XRDP, 3

yum, 2